

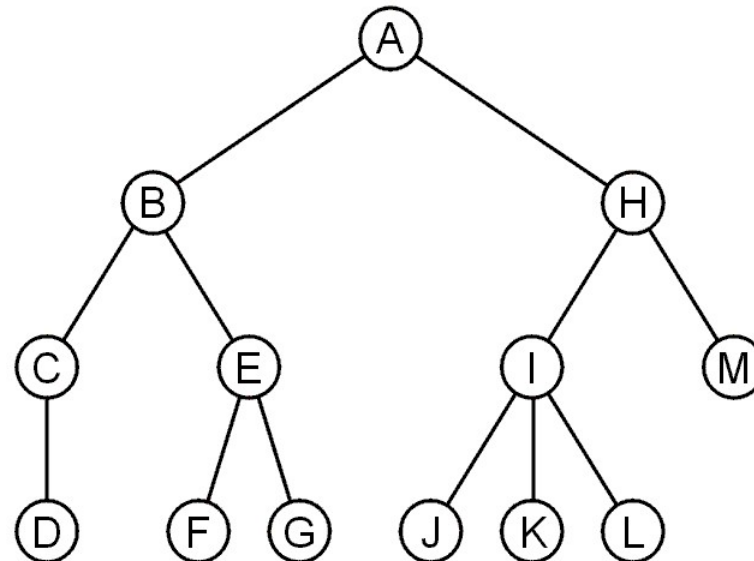
Data Structures (CS2103)

Tree

Prepared By – Ashish K Layek
CST, IEST Shibpur

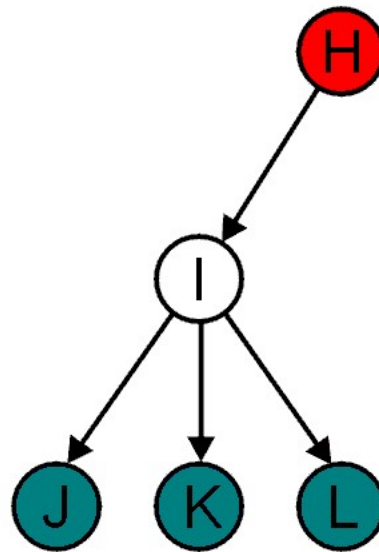
Trees

- ❑ A rooted tree data structure stores information in *nodes*
 - There is a first node, or *root*
 - Each node has variable number of references to successors
 - Each node, other than the root, has exactly one node pointing to it



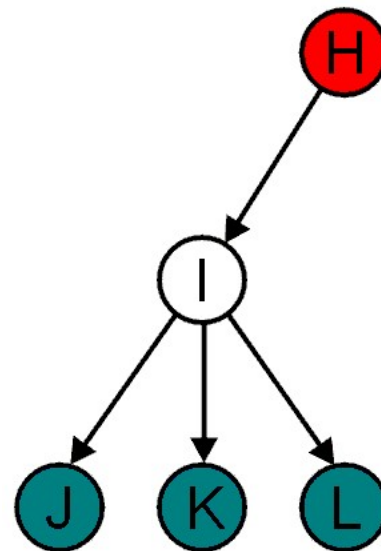
Terminology: Child and Parent

- ❑ All nodes will have zero or more child nodes or *children*
 - I has three children: J, K and L
- ❑ For all nodes other than the root node, there is one parent node
 - H is the parent I



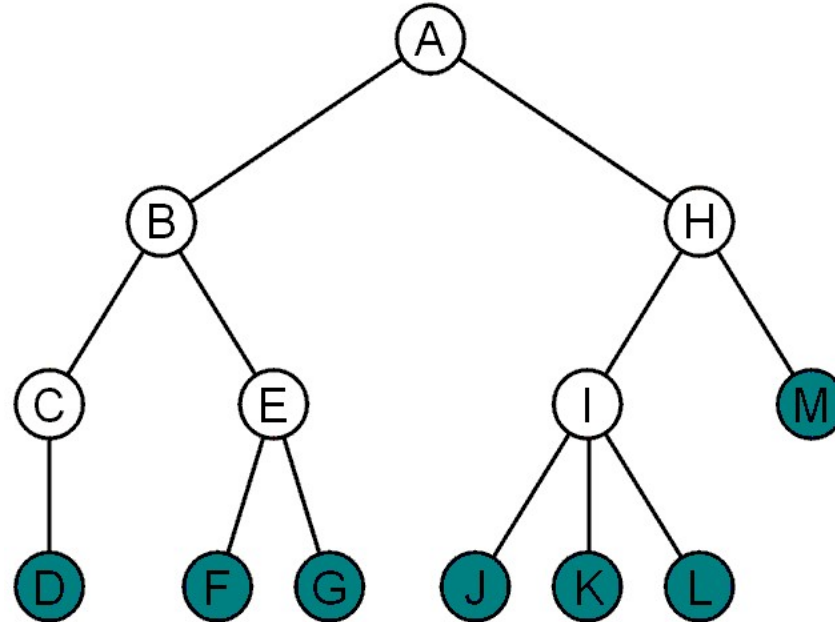
Terminology: Degree

- ❑ The *degree* of a node is defined as the number of its children: $\text{deg}(I) = 3$
- ❑ Nodes with the same parent are *siblings*
 - J, K, and L are siblings



Terminology: Leaf and Internal Node

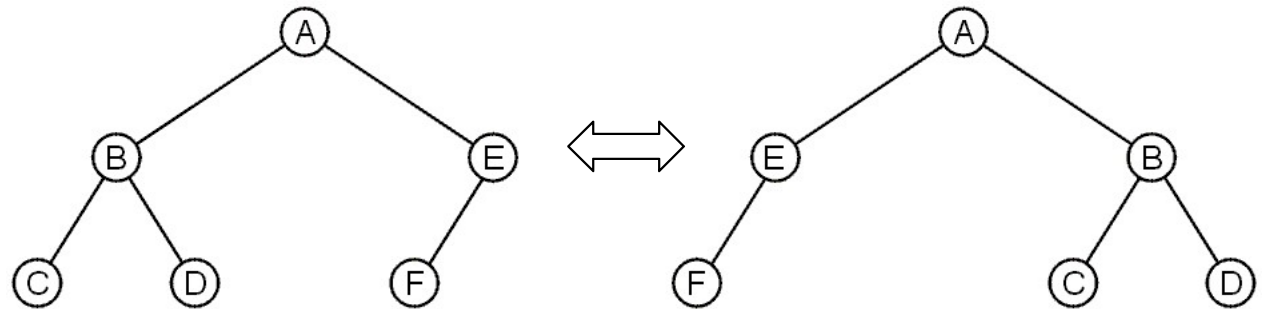
- ❑ Nodes with degree zero are also called *leaf nodes*
- ❑ All other nodes are said to be *internal nodes*, that is, they are internal to the tree



Terminology: Order

- ❑ These trees are equal if the order of the children is ignored

➤ *unordered trees*



- ❑ They are different if order is relevant (*ordered trees*)

- We will usually examine ordered trees (linear orders)
- In a hierarchical ordering, order is not relevant

Terminology: Path

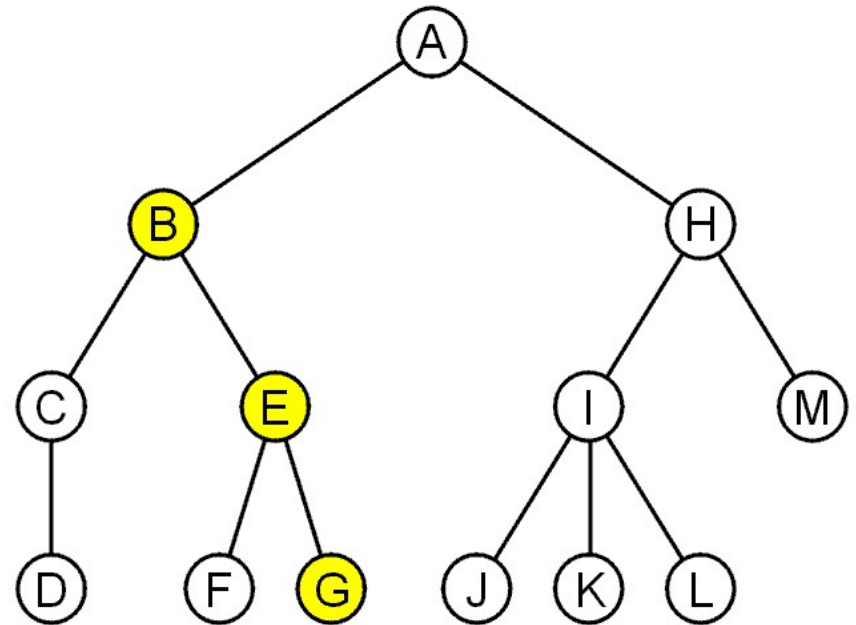
□ A path is a sequence of nodes

$$(a_0, a_1, \dots, a_n)$$

where a_{k+1} is a child of a_k

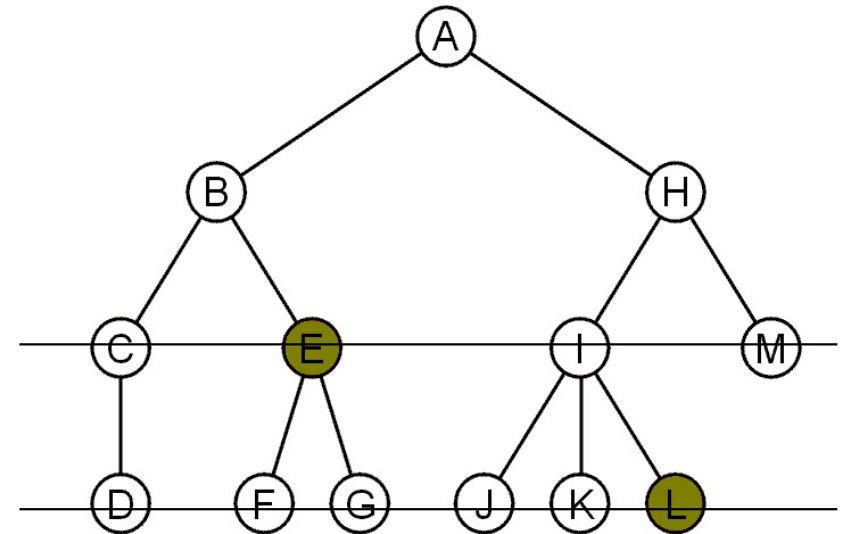
The length of this path is n

E.g., the path (B, E, G)
has length 2



Terminology: Depth and Level

- ❑ The **level** and **depth** are closely related concepts, but they refer to slightly different things depending on perspective
- ❑ For each node in a tree, there exists a unique path from the root node to that node
- ❑ The length of this path is the *depth* of the node, *e.g.*,
 - E has depth 2 and L has depth 3
- ❑ The *level* of a node is defined by initially letting the root be at level ZERO. If a node is at level l , then its children are at level $l + 1$.
- ❑ In most cases, depth = level



Note: Sometimes level of root is considered as 1, depending on convention. In that case depth and level differs in terms of value

Terminology: Height

- ❑ The *height* of a tree is defined as the **maximum depth** of any node within the tree
- ❑ The height of a tree with one node is 0
 - Just the root node
- ❖ For convenience, we define the height of the empty tree to be -1 . (*Empty tree is possible in case of Binary Tree*)

Terminology: Ancestor and Descendent

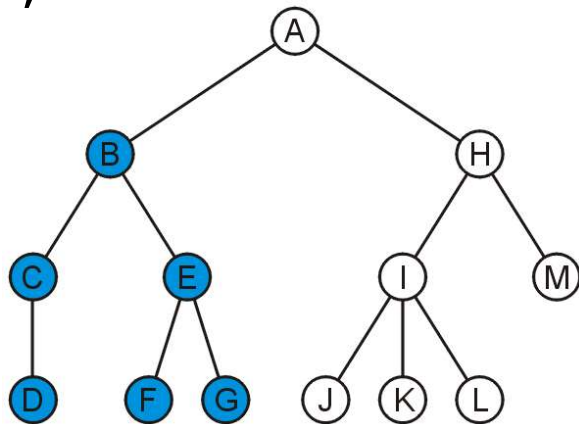
❑ If a path exists from node a to node b :

➤ a is an *ancestor* of b

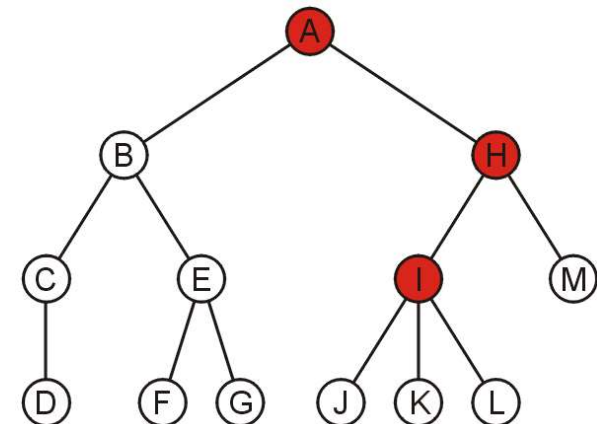
➤ b is a *descendent* of a

❑ The root node is an ancestor of all nodes

The descendants of node B are C, D, E, F, and G:



The ancestors of node I are H, and A:



Tree definition

A tree is a finite set of one or more nodes such that (i) there is a specially designated node called the root; (ii) the remaining nodes are partitioned into $n \geq 0$ disjoint sets $(T_1, T_2, \dots, T_n)$ where each of these sets is a tree. $T_1, T_2, \dots, T_n$ are called the subtrees of the root.

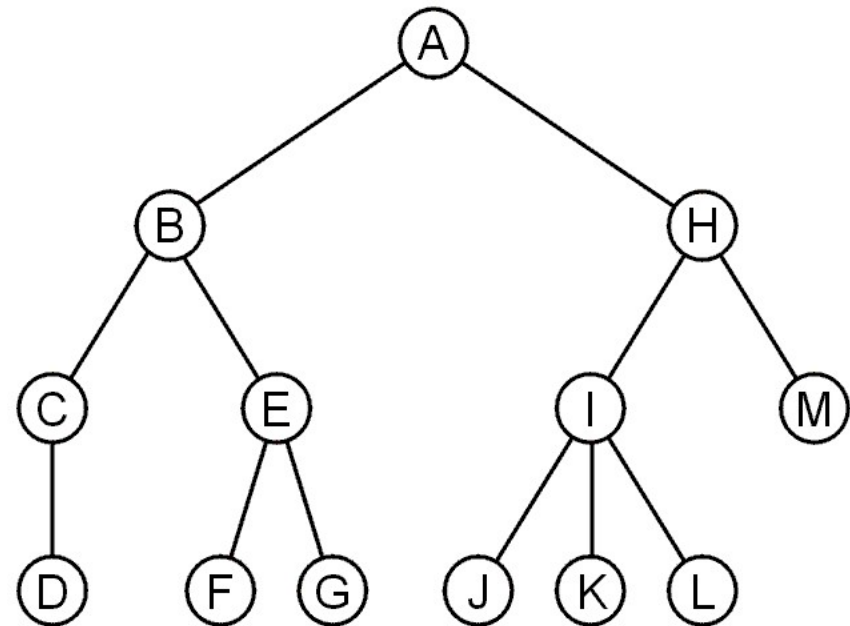
Let us look carefully at the definition and note down the important points.

1. the definition is recursive.
2. the set is finite and must have at least one node for it to be tree.
3. an empty set is not considered to be a tree.
4. the condition that $T_1, T_2, \dots, T_n$ be disjoint sets prohibits subtrees from ever connecting together.
5. every node in the tree is root of some subtree.

Abstract Trees

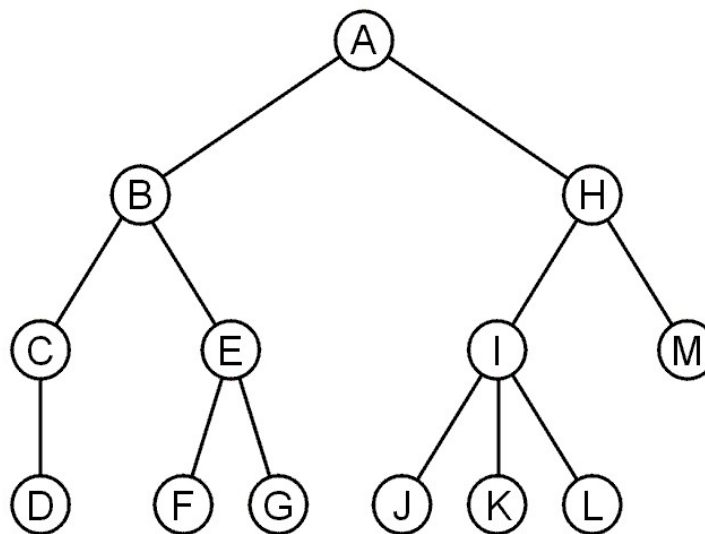
- ❑ An abstract tree (or abstract hierarchy) does not restrict the number of nodes
 - In this tree, the degrees vary:

Degree	Nodes
0	D, F, G, J, K, L, M
1	C
2	A, B, E, H
3	I



Abstract Trees: Design

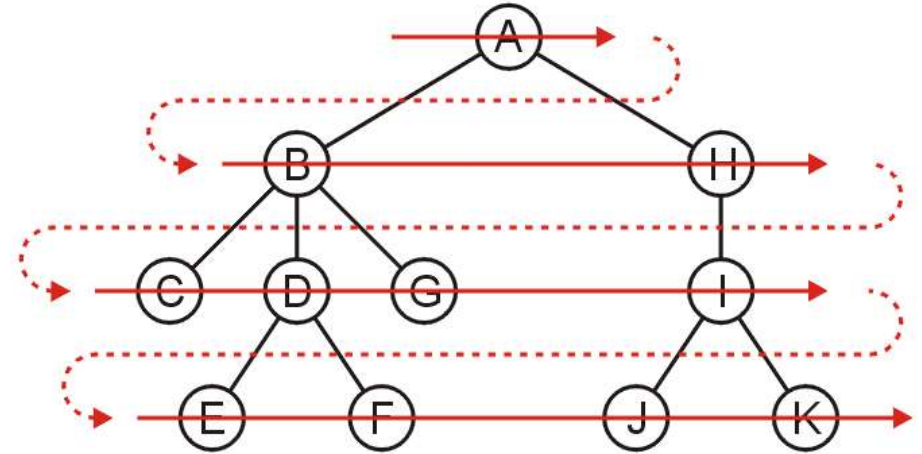
- ❑ We implement an abstract tree or hierarchy by using a structure that:
 - Stores a value
 - Stores the children in a linked-list



Tree Traversal (for your study)

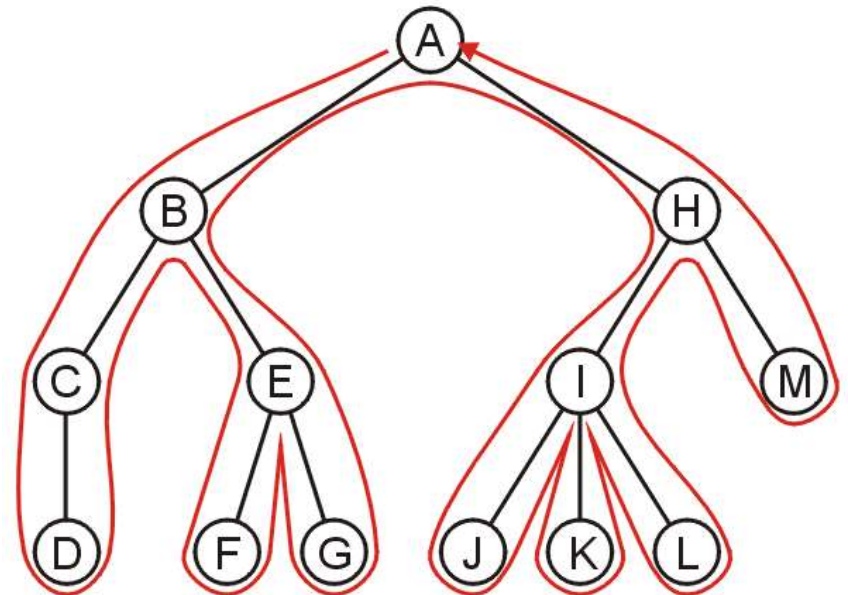
❑ Breadth-first traversal

- Visits all nodes at depth k before proceeding onto depth $k + 1$



❑ Depth-first traversal

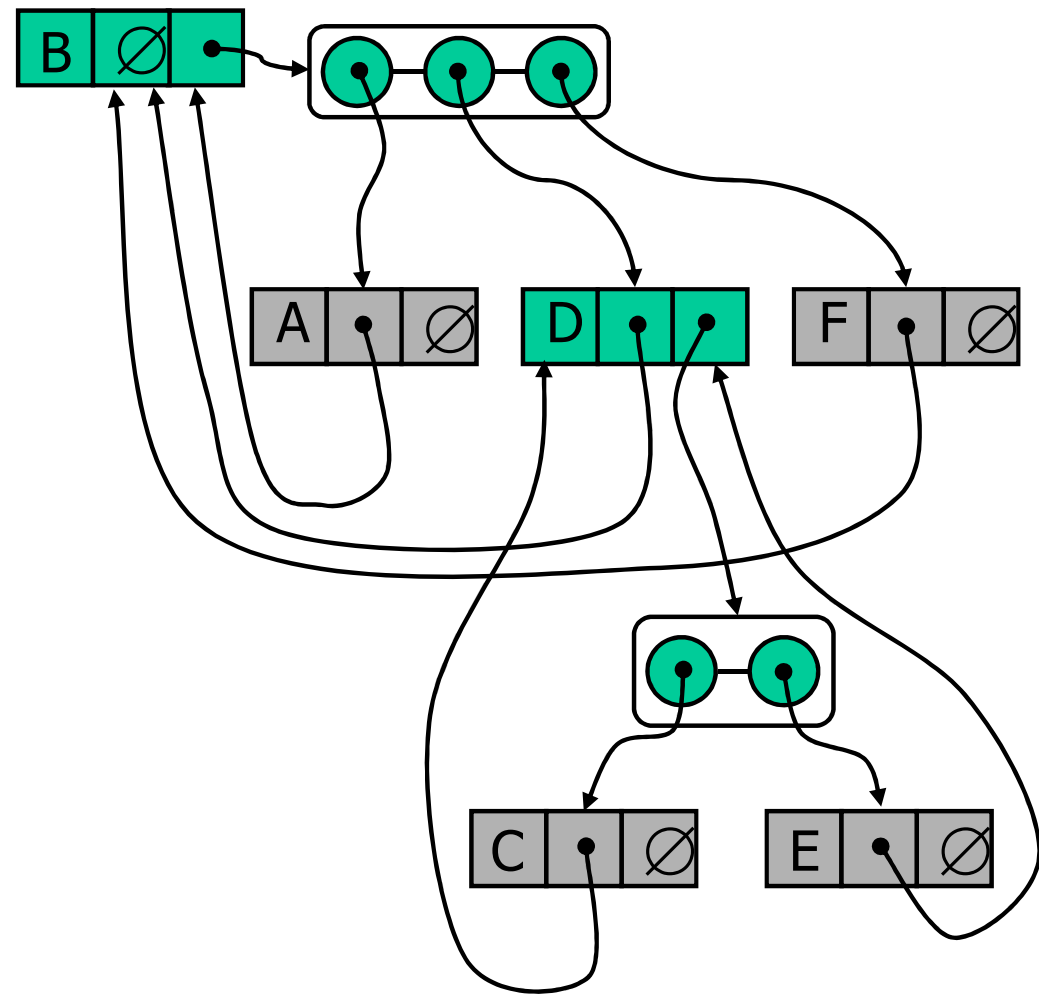
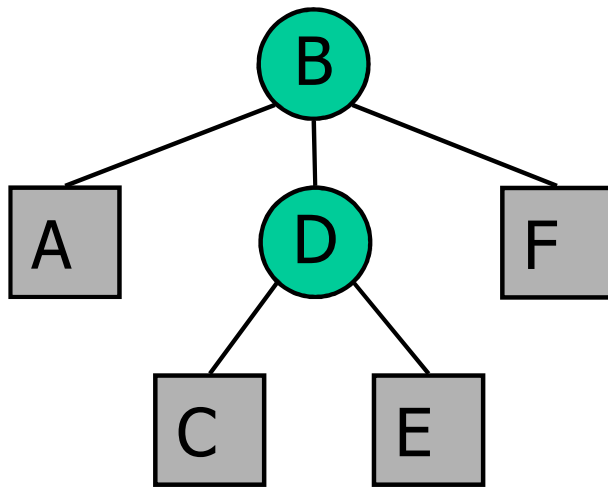
- At any node, we proceed to the first child that has not yet been visited
- Or, if we have visited all the children (of which a leaf node is a special case), we backtrack to the parent and repeat this decision making process



Linked Structure for Trees

□ A node is represented by an structure storing

- Element
- Parent node
- Sequence of children nodes



Linked Structure for Trees

Contd ...

```
// forward declaration
struct childNode;

struct treeNode {
    // data field
    char info;
    // reference to parent
    struct treeNode * parent;
    // list of childs
    struct childNode * childs;
};

struct childNode {
    struct treeNode child; // * ??
    struct childNode * next;
};
```

```
// An alternative approach
with limitations

struct treeNode {
    // data field
    char info;
    // reference to parent
    struct treeNode * parent;
    // list of childs can be
    allocated according to the
    tree structure - like number
    of childs.

    struct treeNode ** childs;
};
```

Binary Tree

Motivation

- ❑ The arbitrary number of children in general trees is often unnecessary—many real-life trees are restricted to two branches
 - Expression trees using binary operators
 - An ancestral tree of an individual, parents, grandparents, *etc.*
 - Phylogenetic trees
 - Lossless encoding algorithms
- ❖ There are also issues with general trees:
 - There is no natural order between a node and its children

Binary Tree definition

A binary tree is a finite set of nodes which is either empty or consists of a root and two disjoint binary trees called the left subtree and the right subtree.

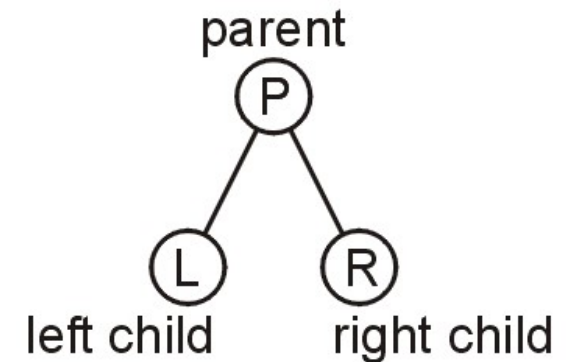
Few important observations to note from the definition

1. The definition is recursive.
 2. Unlike tree, binary tree can be empty.
 3. Order of the subtree does matter here, i.e., we distinguish between left subtree and right subtree.
-

Explanation

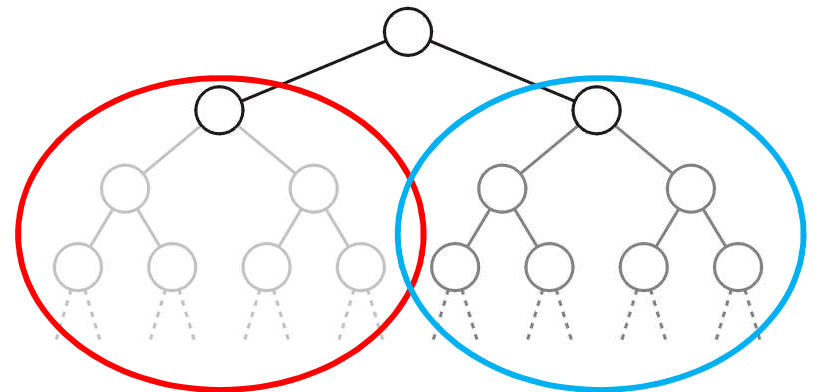
❑ A binary tree is a restriction where each node has exactly two children:

- Each child is either empty or another binary tree
- This restriction allows us to label the children as *left* and *right* subtrees



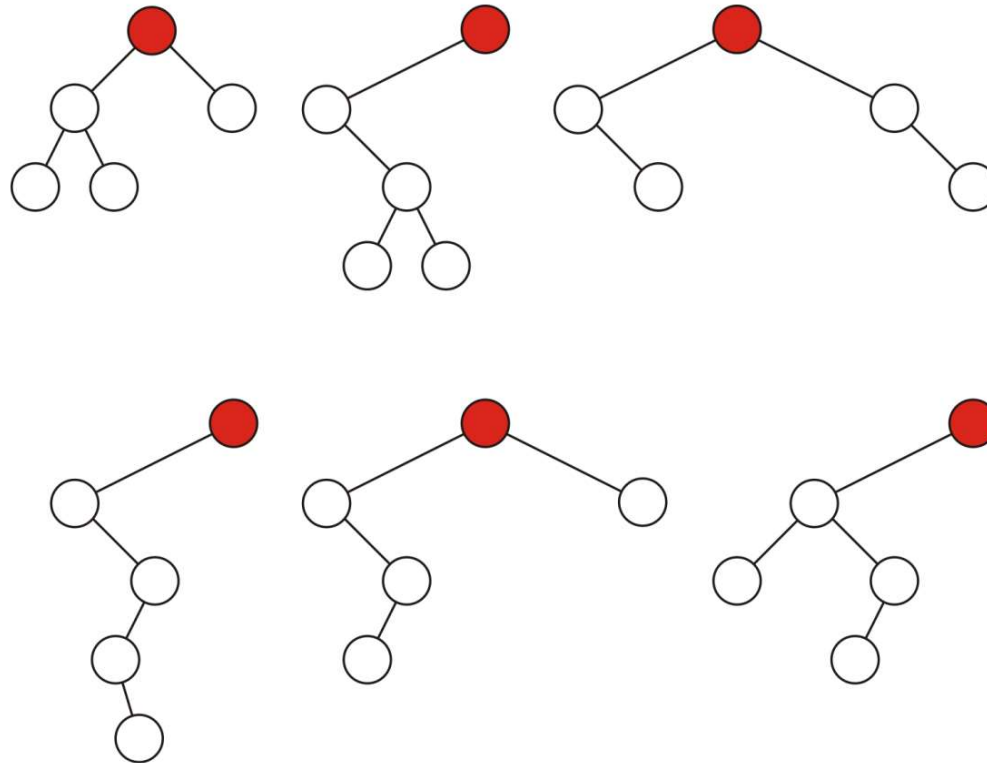
❑ We will also refer to the two sub-trees as

- The left-hand sub-tree, and
- The right-hand sub-tree



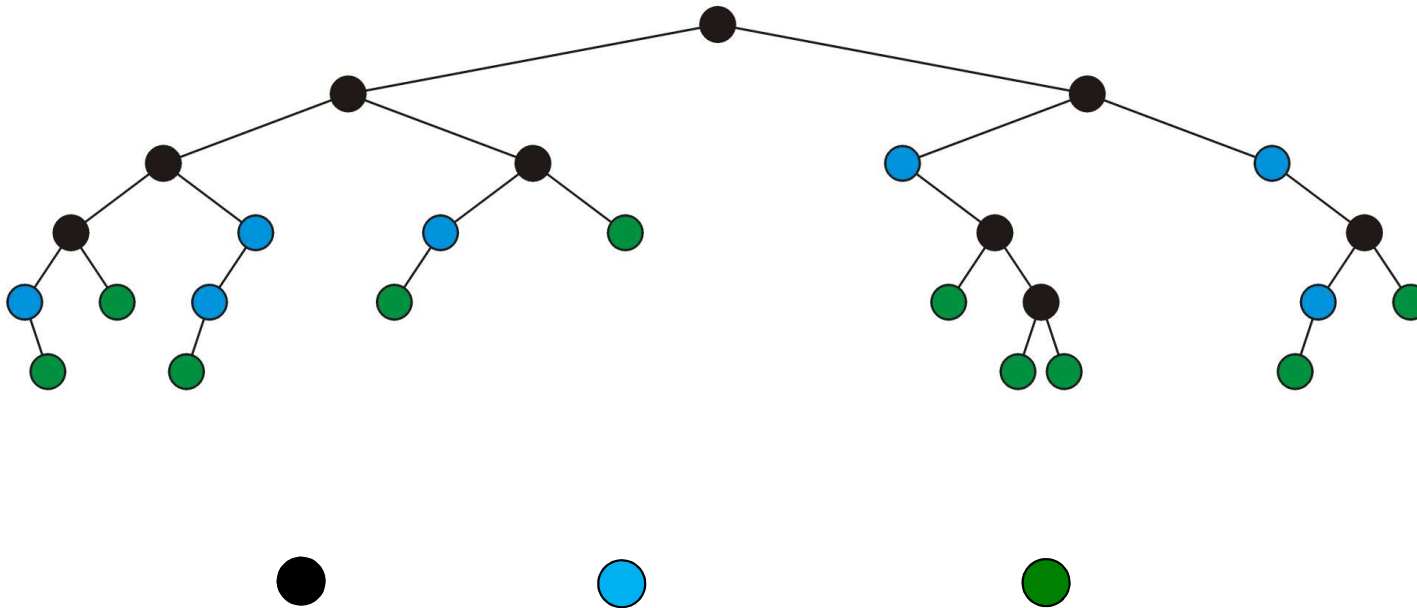
Example

❑ Sample variations on binary trees with five nodes:



Definition: Full Node

- ❑ A *full* node is a node where both the left and right sub-trees are non-empty trees



Legend:

full nodes

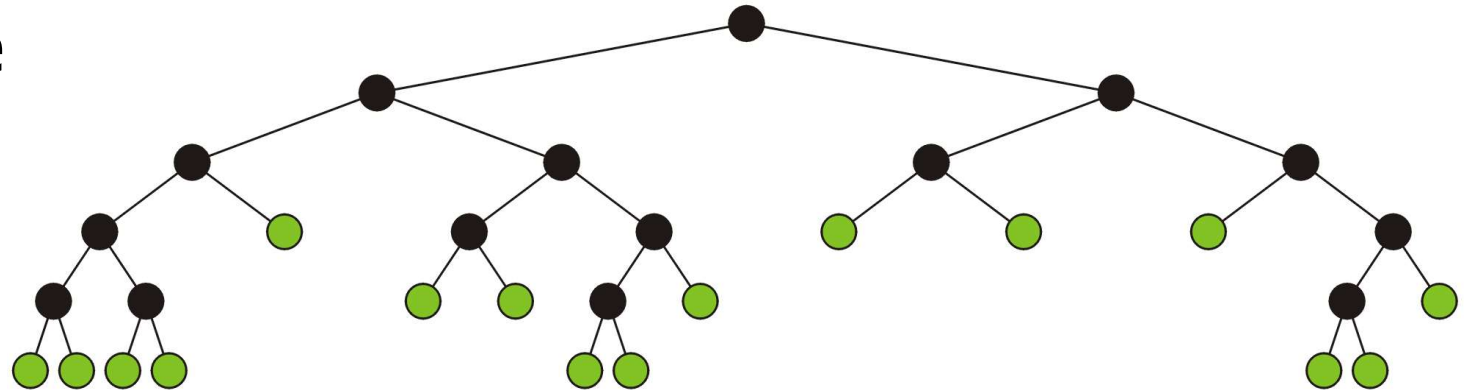
neither

leaf nodes

Definition: Full / Strict Binary Tree

❑ A *full binary tree* (also called *strict binary tree*) is where each node is:

- A full node,
- or a leaf node



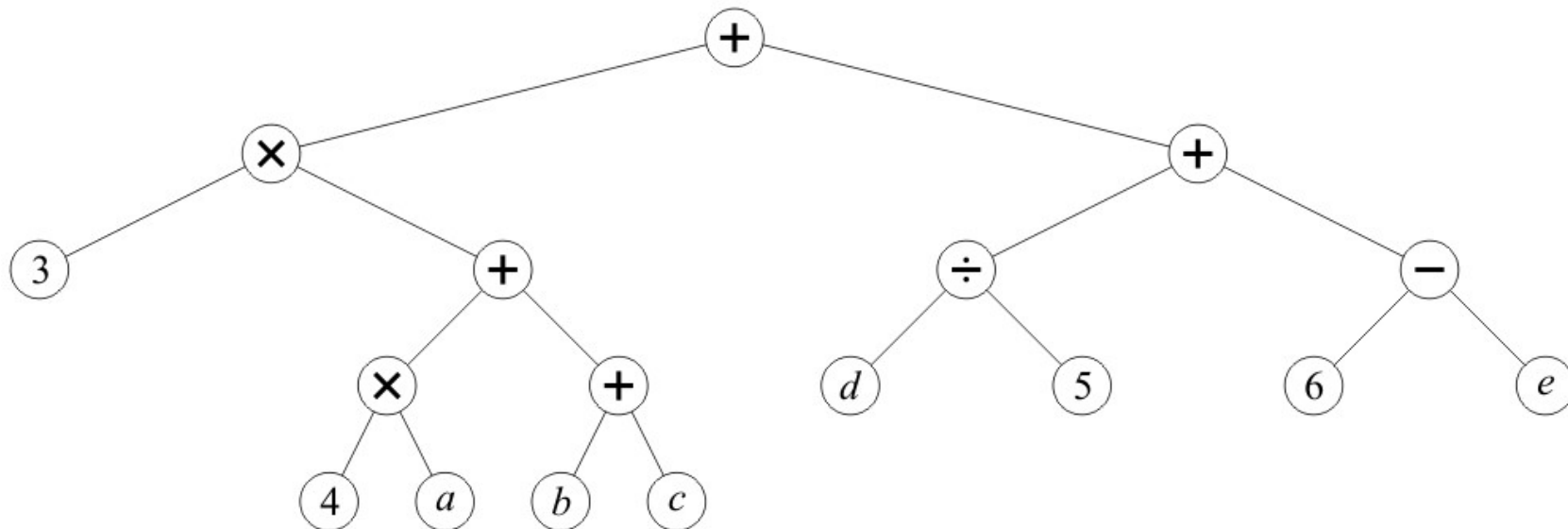
❖ These have applications in

- Expression trees
- Huffman encoding

Application: Expression Trees

❑ Any basic mathematical expression containing binary operators may be represented using a binary tree

❖ For example, $3(4a + b + c) + d/5 + (6 - e)$

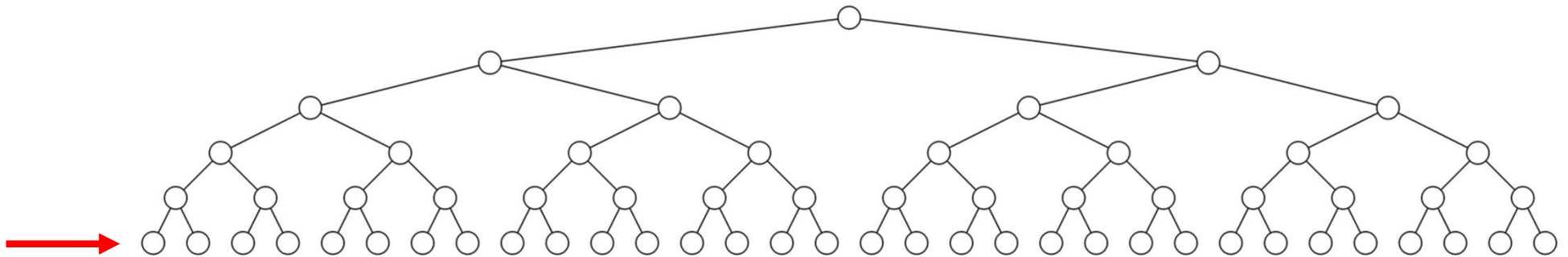


To be discussed more ...

Definition: Perfect Binary Tree

□ Standard definition:

- A perfect binary tree of height h is a binary tree where
 - ✓ All leaf nodes have the same depth h
 - ✓ All other nodes are full

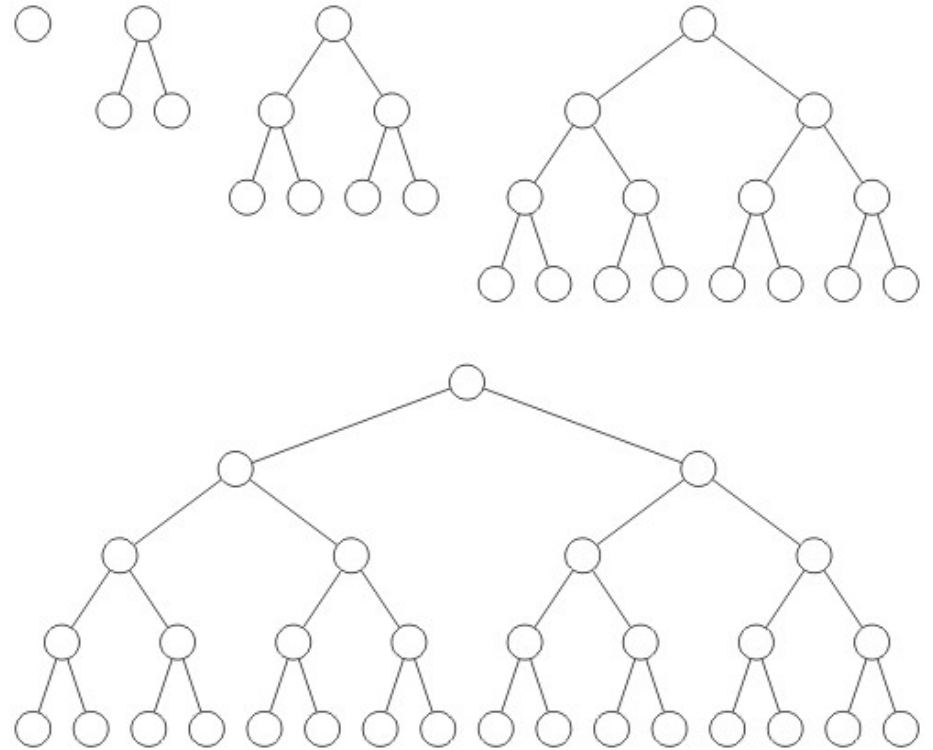


□ Recursive definition:

- A binary tree of height $h = 0$ is perfect
- A binary tree with height $h > 0$ is a perfect if both subtrees are perfect binary trees of height $h - 1$

Perfect Binary Tree Examples

❑ Perfect binary trees of height $h = 0, 1, 2, 3$ and 4



❖ Perfect binary trees are considered to be the *ideal* case.

➤ Aim is to find trees which are as close as possible to perfect binary trees

Perfect Binary Tree: $(2^{h+1} - 1)$ Nodes

□ Theorem-1: A perfect binary tree of height h has $2^{h+1} - 1$ nodes

❖ We will use mathematical induction:

1. Show that it is true for $h = 0$
2. Assume it is true for an arbitrary h
3. Show that the truth for h implies the truth for $h + 1$

Step-1: The base case:

When $h = 0$ we have a single node $n = 1$

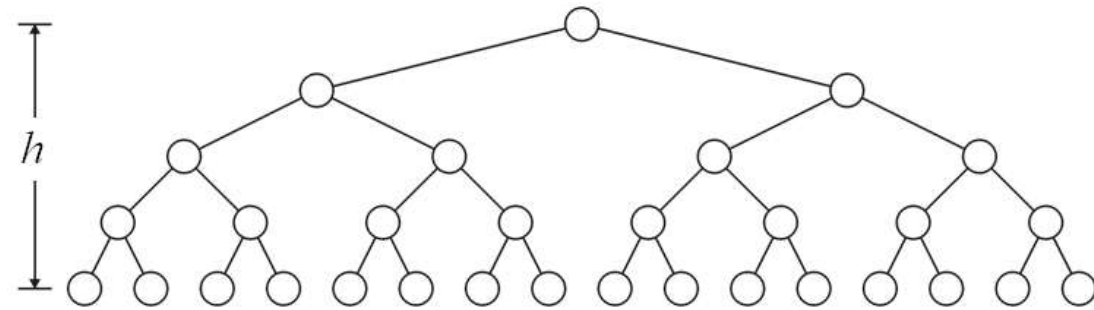
The formula is correct: $2^{0+1} - 1 = 1$

$(2^{h+1} - 1)$ Nodes

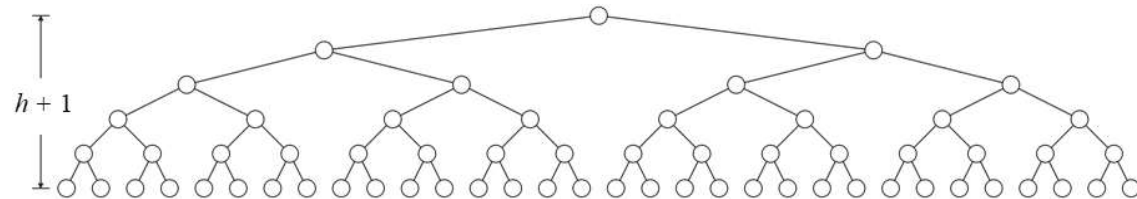
Cont...

Step-2: The inductive step:

- If the height of the tree is h , then assume that the number of nodes is $n = 2^{h+1} - 1$



- We must show that a tree of height $h + 1$ has $n = 2^{(h+1)+1} - 1 = 2^{h+2} - 1$ nodes



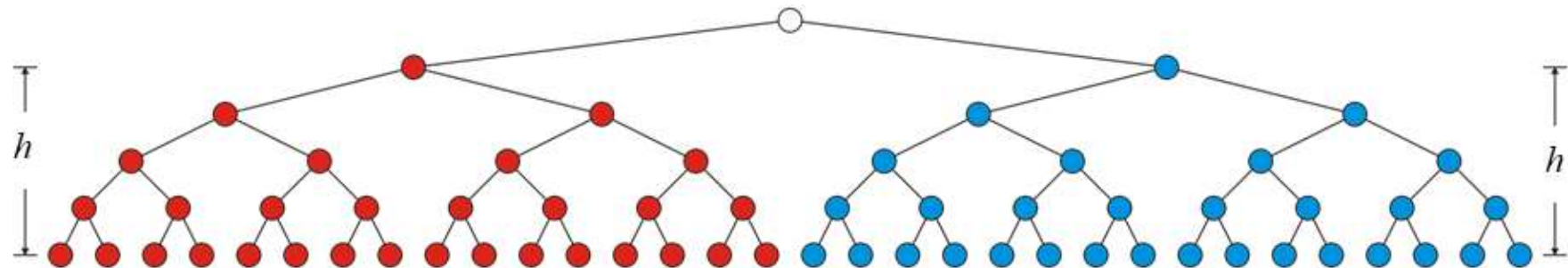
$(2^{h+1} - 1)$ Nodes

Cont...

Step-3: Using the recursive definition, both sub-trees are perfect trees of height h

- By assumption, each sub-tree has $2^{h+1} - 1$ nodes
- Therefore the total number of nodes is

$$(2^{h+1} - 1) + 1 + (2^{h+1} - 1) = 2^{h+2} - 1$$



- ❖ Consequently, The statement is true for $h = 0$ and the truth of the statement for an arbitrary h implies the truth of the statement for $h + 1$.
 - Therefore, by the process of mathematical induction, the statement is true for all $h \geq 0$

Perfect Binary Tree: Logarithmic Height

□ Theorem-2: A perfect binary tree with n nodes has height $\log(n + 1) - 1$

❖ Proof

Solving $n = 2^{h+1} - 1$ for h :

$$n + 1 = 2^{h+1}$$

$$\log(n + 1) = h + 1$$

$$h = \log(n + 1) - 1$$

Perfect Binary Tree: 2^h Leaf Nodes

□ Theorem-3: A perfect binary tree with height h has 2^h leaf nodes

❖ Proof (by induction):

- When $h = 0$, there is $2^0 = 1$ leaf node.
- Assume that a perfect binary tree of height h has 2^h leaf nodes and observe that both sub-trees of a perfect binary tree of height $h + 1$ have 2^h leaf nodes.
- Therefore, the perfect tree of height $h + 1$ must have $2 \cdot 2^h = 2^{h+1}$ leaf nodes.

Perfect Binary Tree: Over half of all nodes are Leaf Nodes

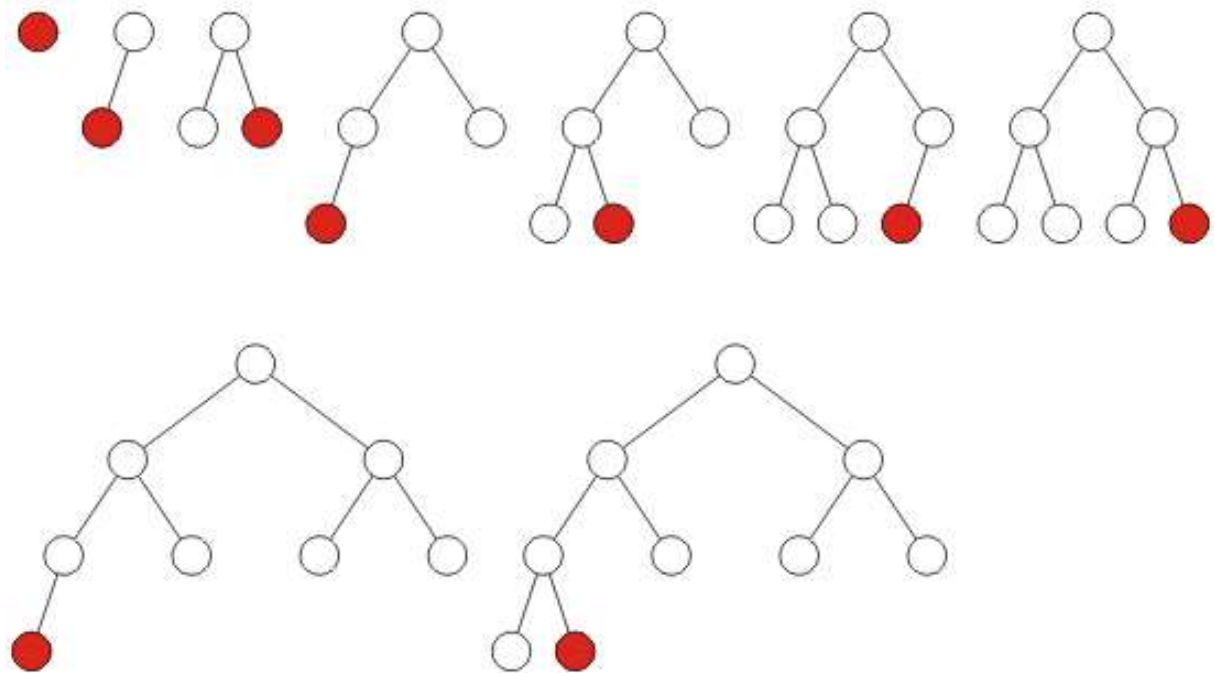
- ❑ Consequence : Over half of all the nodes in a perfect binary tree are leaf nodes.
- ❖ From Theorem-3 and Theorem-1, we have the number of leaf nodes and the total number of nodes. Calculating the ratio, we have

$$\frac{2^h}{2^{h+1} - 1} > \frac{2^h}{2^{h+1}} = \frac{1}{2}.$$

- ❖ Conclusion: Over half all nodes are leaf nodes:

Definition: Complete Binary Tree

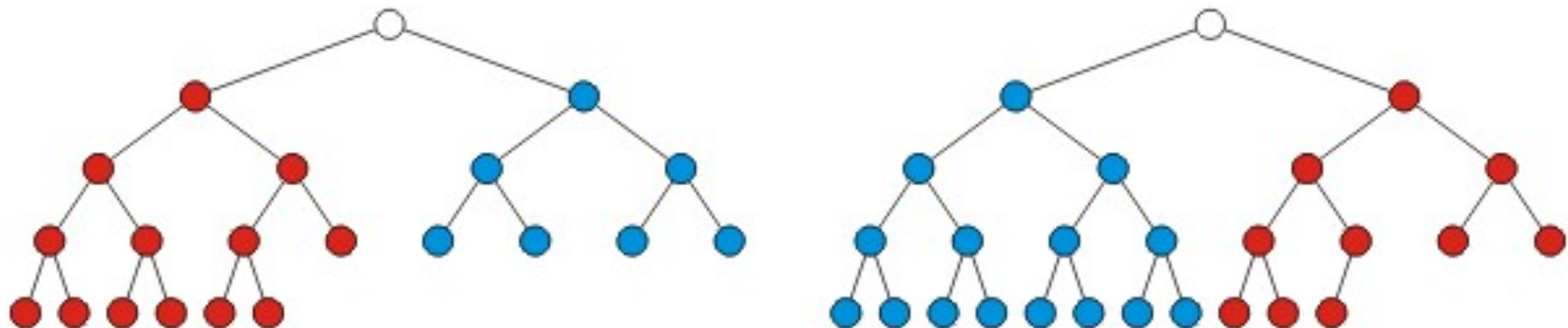
- ❑ A complete binary tree filled at each depth from left to right.
- ❑ The order is identical to that of a breadth-first traversal.



- ❖ The height of a complete binary tree with n nodes is $h = \lfloor \log(n) \rfloor$

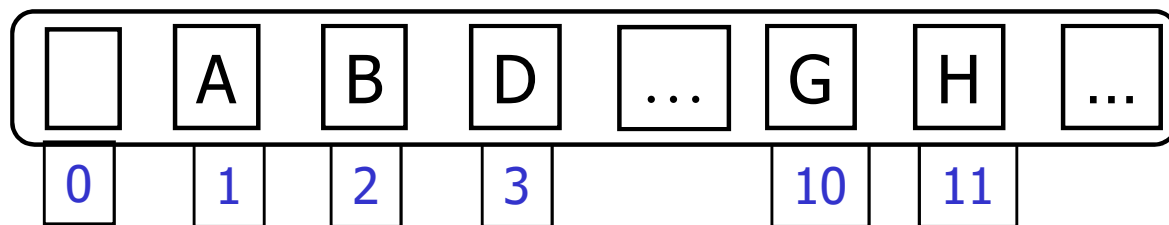
Complete Binary Tree: Recursive Definition

- ❑ Recursive definition: a binary tree with a single node is a complete binary tree of height $h = 0$ and a complete binary tree of height h is a tree where either:
 - The left sub-tree is a **complete binary tree** of height $h - 1$ and the right sub-tree is a **perfect binary tree** of height $h - 2$, or
 - The left sub-tree is **perfect binary tree** with height $h - 1$ and the right sub-tree is **complete binary tree** with height $h - 1$

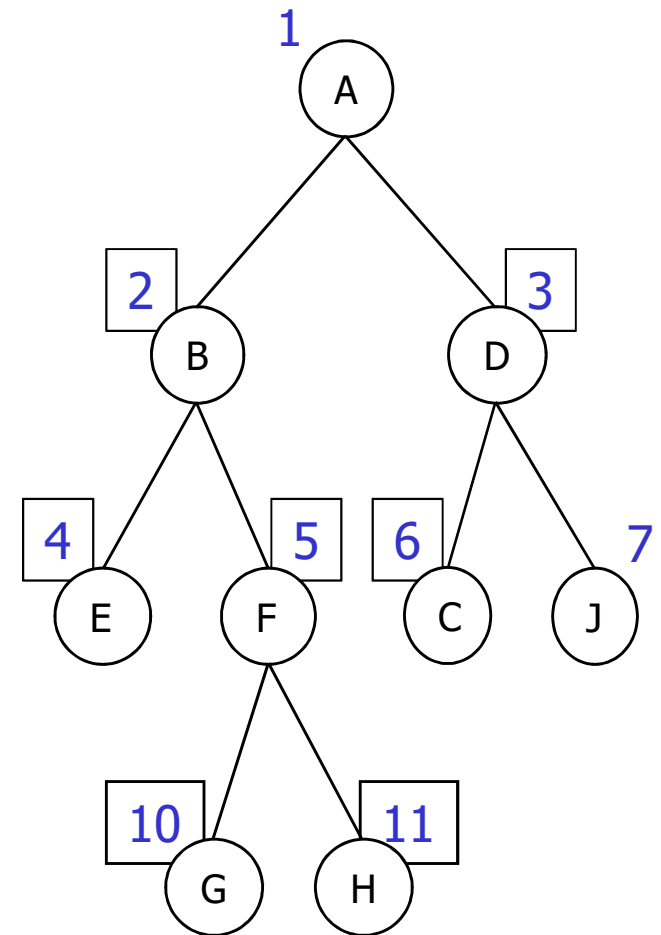


Array-Based Representation of Binary Trees

- ❑ Nodes are stored in an array (*leaving first element unused in case base index of the array is 1*)



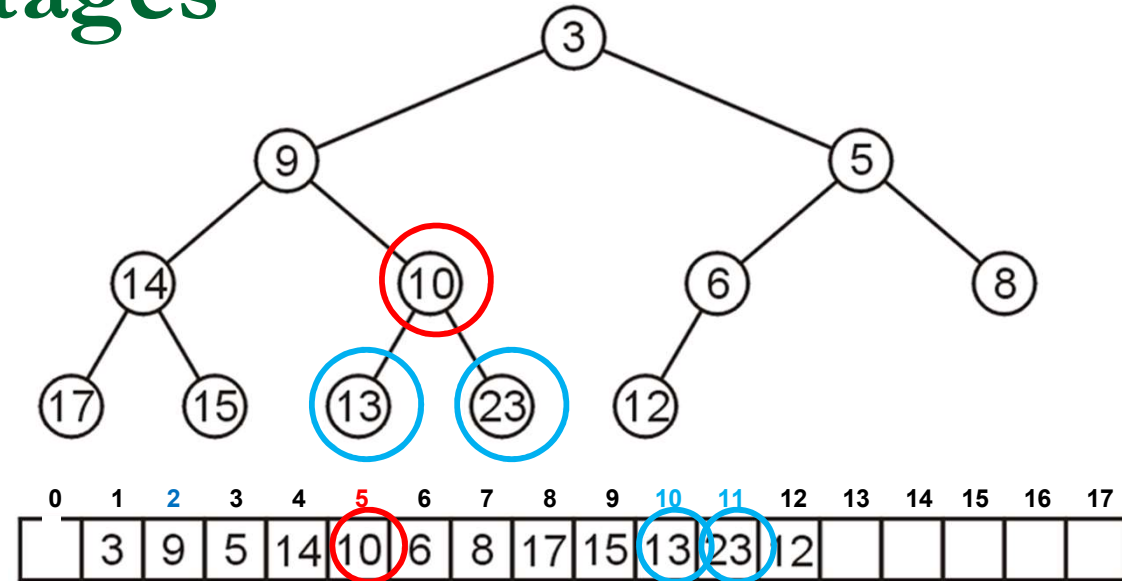
- ❑ The left child of node i can be found at position $2i$
- ❑ The right child of node i can be found at position $2i + 1$
- ❑ The parent of node i can be found at position $i/2$



Array storage: Advantages

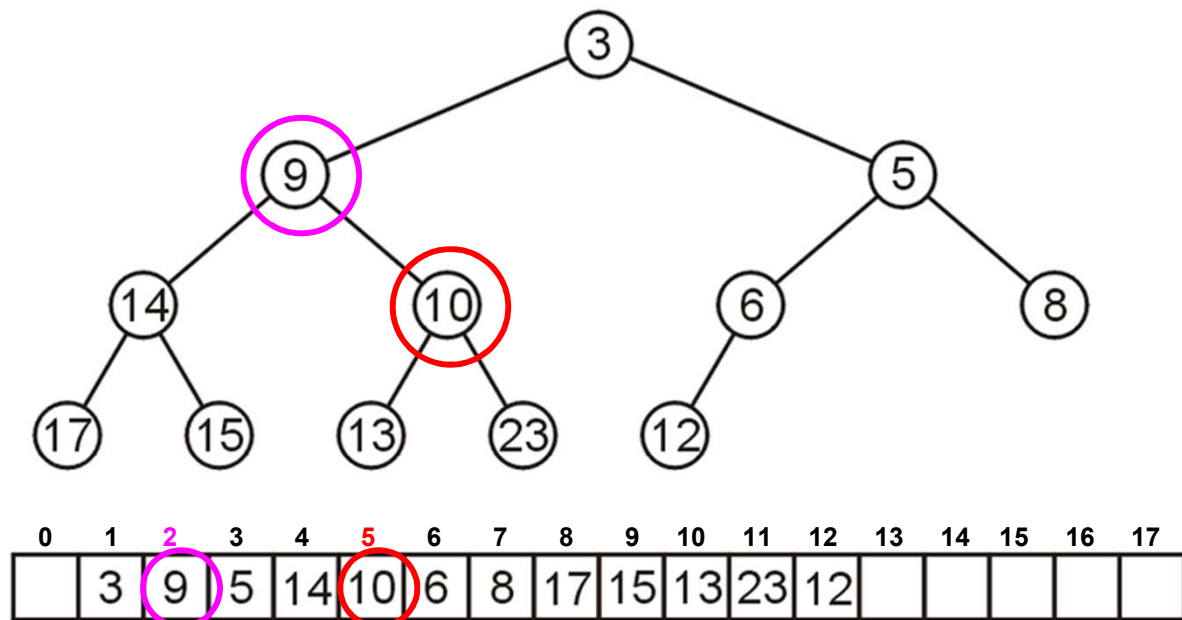
❑ For example, node 10 has index **5**:

- Its children 13 and 23 have indices **10** and **11**, respectively



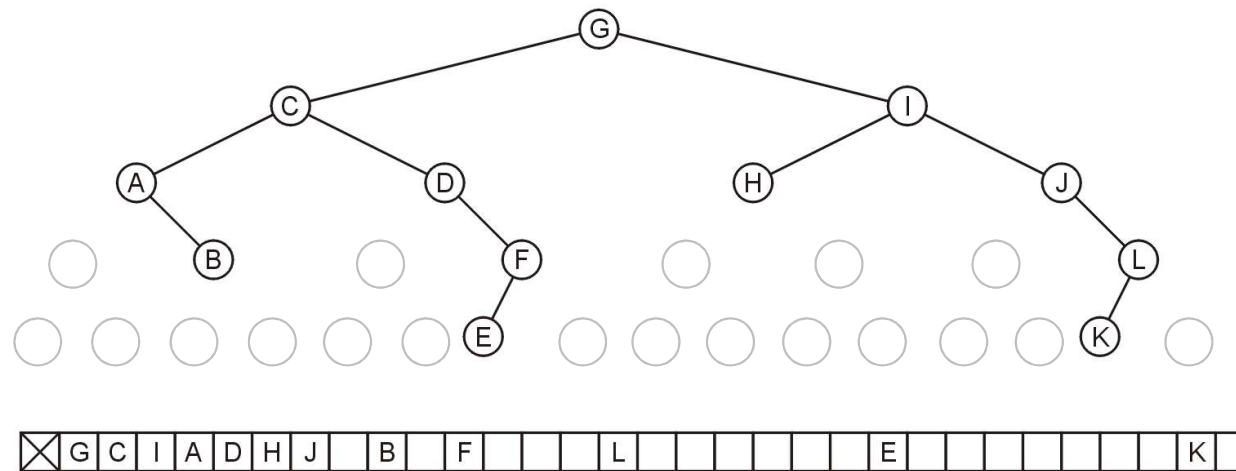
- Its parent is node 9 with index $5/2 = 2$

❖ Conclusion: Constant time access to parent or child



Array storage: Disadvantages

- ❑ There is a significant potential for a lot of wasted memory
 - Consider this tree with 12 nodes would require an array of size 32
 - ✓ Adding a child to node K doubles the required memory



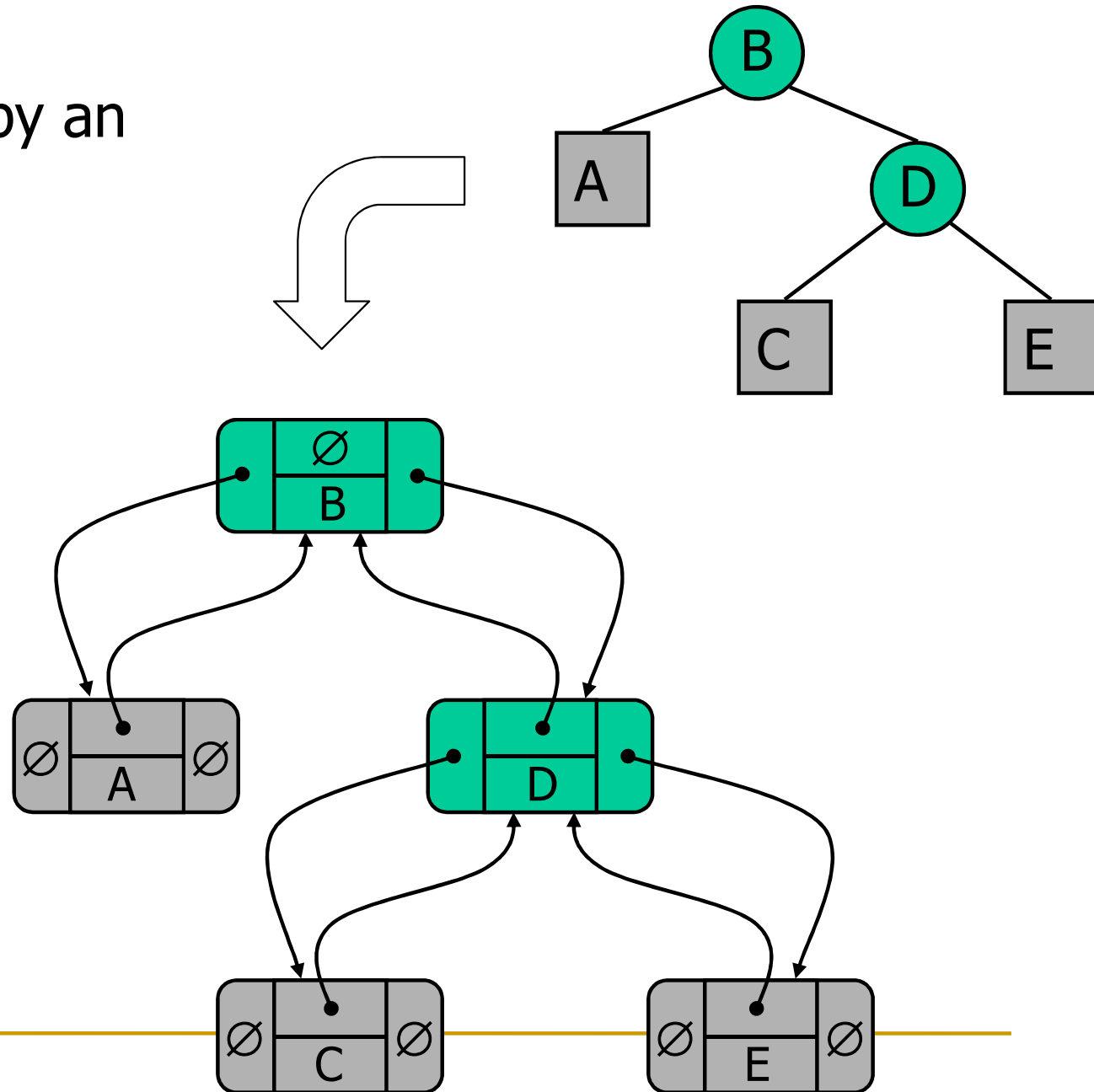
❖ Conclusion: Array based binary tree representation is suitable only for Perfect and Complete Binary Tree

Linked Node Representation for Binary Trees

❑ A node is represented by an structure storing

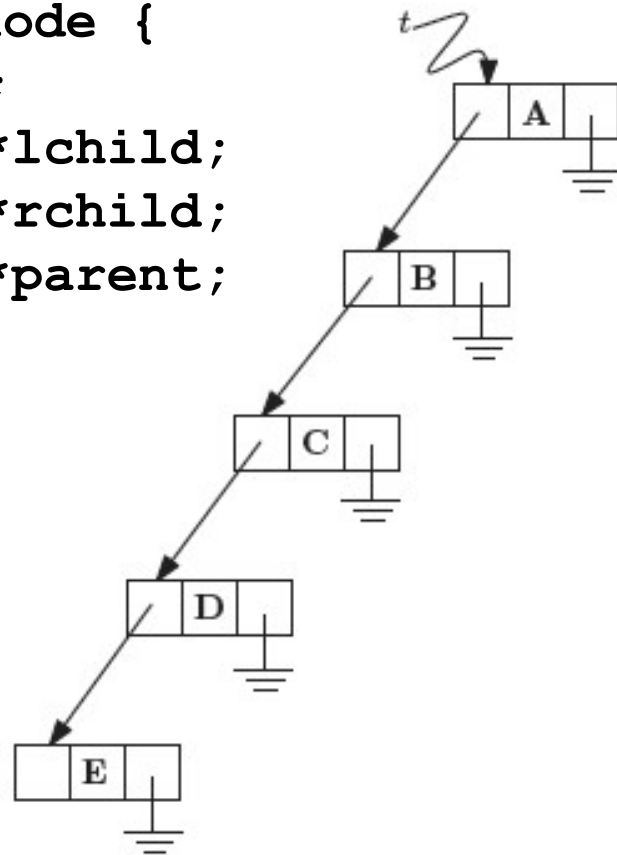
- Element
- Parent node
- Left child node
- Right child node

```
typedef struct node {  
    char element;  
    struct node *lchild;  
    struct node *rchild;  
    struct node *parent;  
} treeNode;
```

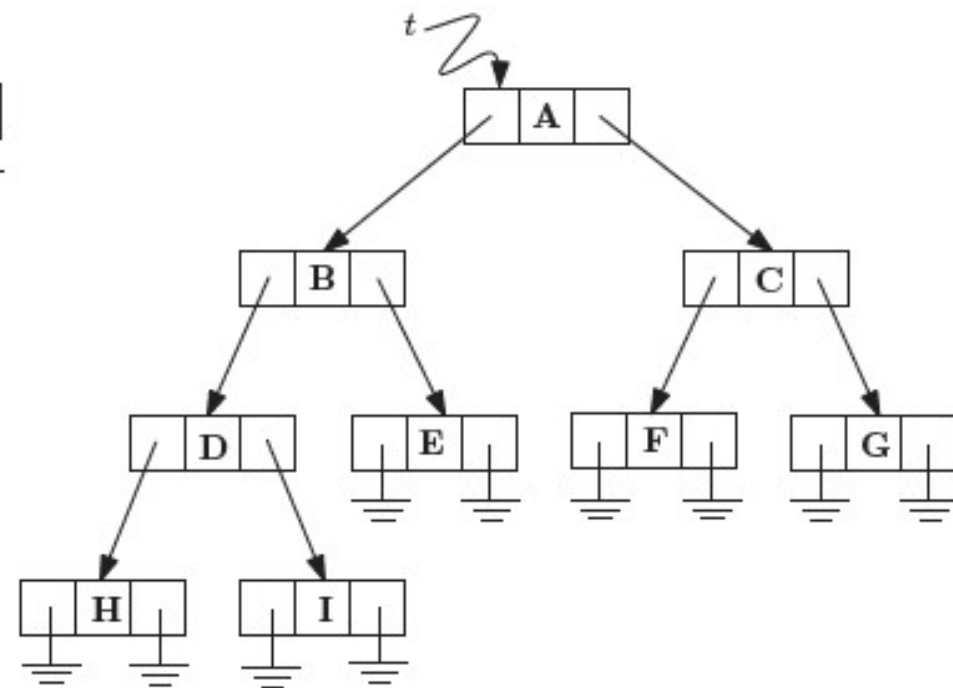


Examples

```
typedef struct node {  
    char element;  
    struct node *lchild;  
    struct node *rchild;  
    struct node *parent;  
} treenode;
```



(a)



(b)

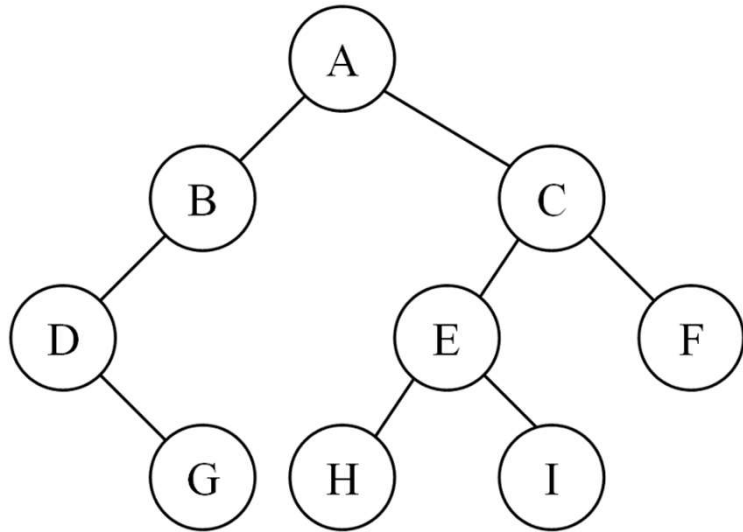
Note: Arrows to Parent nodes are not shown in these diagrams

Binary Tree Traversal

- ❑ When traversing a binary tree we want to treat each node and its subtrees in the same fashion.
 - ❑ If we let L, D, R stand for moving left, printing the data, and moving right when at a node then there are six possible combinations of traversal: LDR, LRD, DLR, DRL, RDL, and RLD.
 - ❑ If we adopt the convention that we traverse **left before right** then only three traversals remain: **LDR, LRD and DLR**. To these we assign the names **inorder, postorder, and preorder**
 - ❑ There is a natural correspondence between these traversals and producing the **infix, postfix and prefix** forms of an expression.
-

Binary Tree traversal

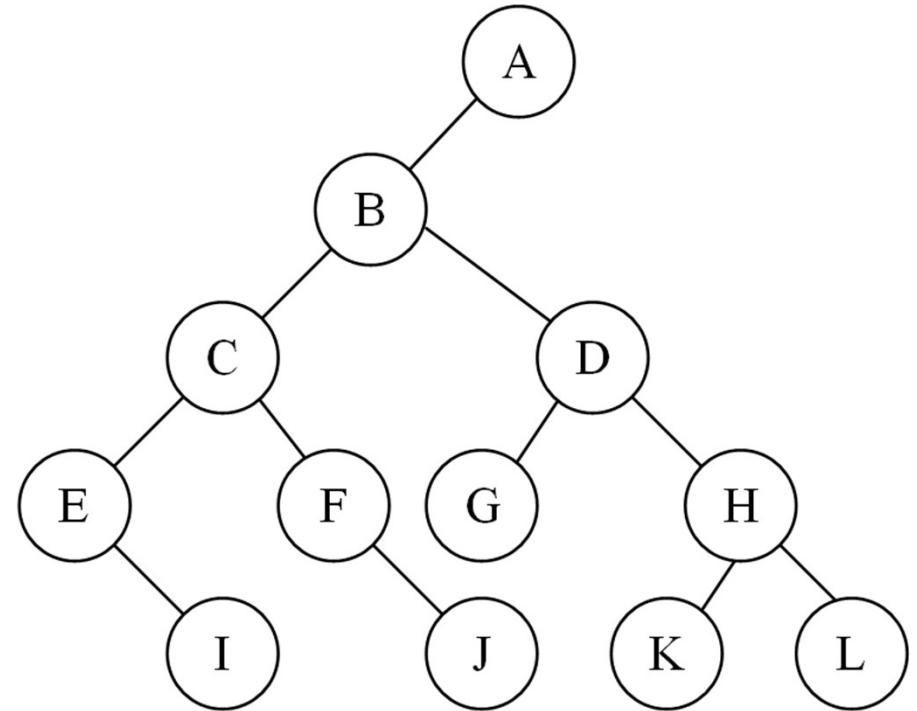
- ❑ Preorder → D L R
- ❑ Inorder → L D R
- ❑ Postorder → L R D



Preorder: A B D G C E H I F

Inorder: D G B A H E I C F

Postorder: G D B H I E F C A



Preorder: A B C E I F J D G H K L

Inorder: E I C F J B G D K H L A

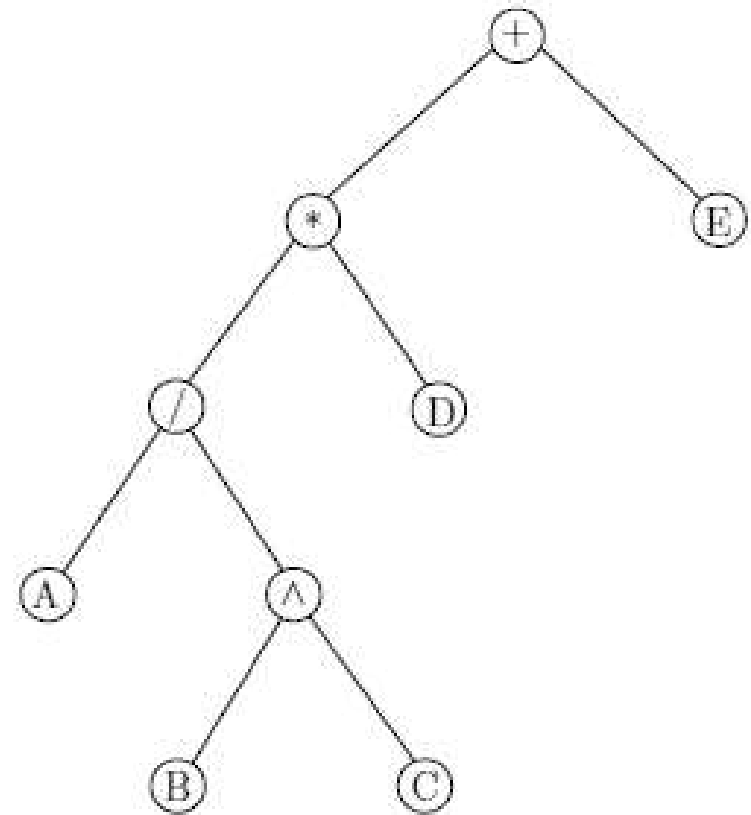
Postorder: I E J F C G K L H D B A

Inorder Traversal

1. Traverse the left subtree in inorder
2. Visit the root.
3. Traverse the right subtree in inorder

```
void inorder(treenode *t) {  
    if(t != NULL) {  
        inorder(t->lchild);  
        printf("%c ", t->element);  
        inorder(t->rchild);  
    }  
}
```

Output: A / B ^ C * D + E

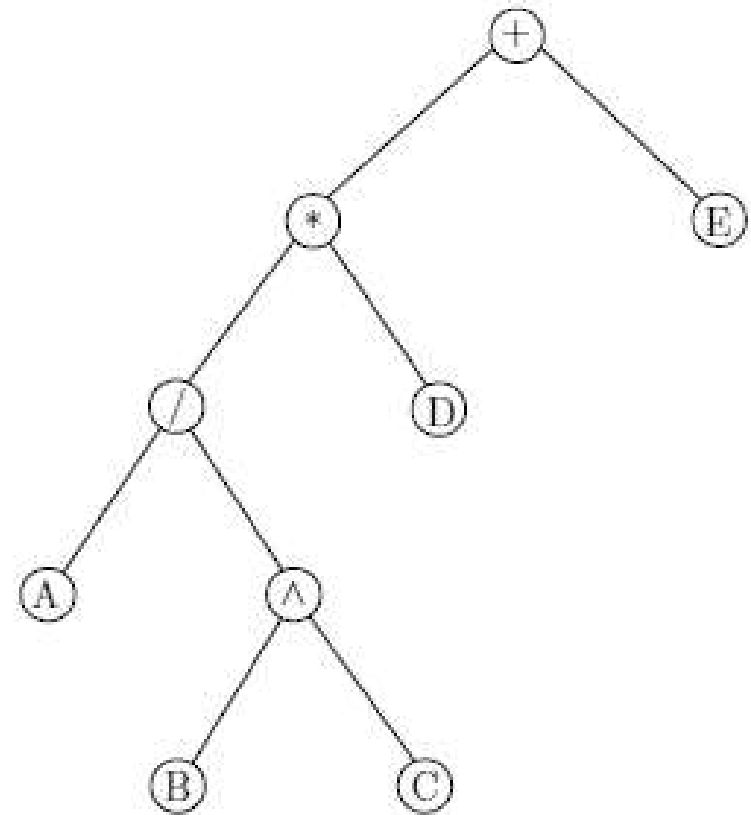


Preorder Traversal

1. Visit the root
2. Traverse the left subtree in preorder
3. Traverse the right subtree in preorder

```
void preorder(treenode *t) {  
    if(t != NULL) {  
        printf("%c ", t->element);  
        preorder(t->lchild);  
        preorder(t->rchild);  
    }  
}
```

Output: + * / A ^ B C D E

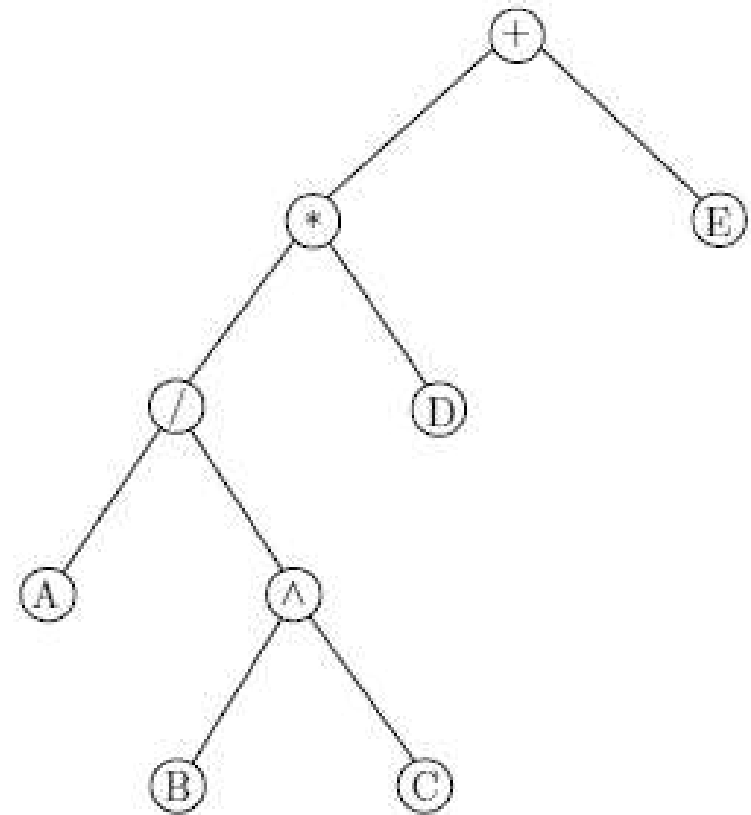


Postorder Traversal

1. Traverse the left subtree in postorder
2. Traverse the right subtree in postorder
3. Visit the root

```
void postorder(treenode *t) {  
    if(t != NULL) {  
        postorder(t->lchild);  
        postorder(t->rchild);  
        printf("%c ", t->element);  
    }  
}
```

Output: A B C ^ / D * E +



Nonrecursive algorithm for traversal

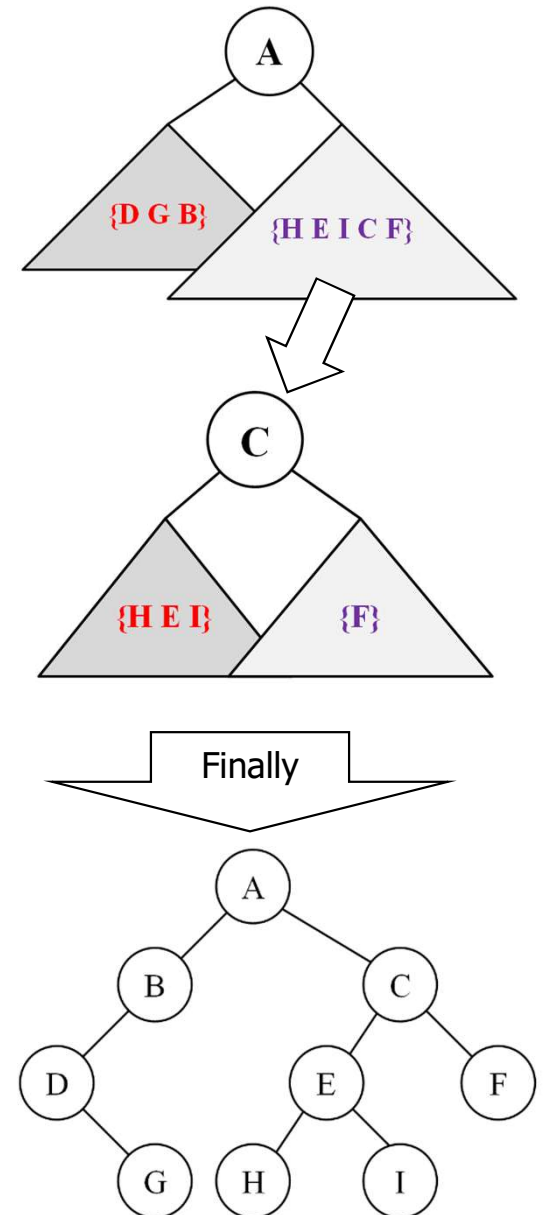
For your study

Observation

- ❑ It can be observed that different binary trees with the same set of values may produce the same traversal (inorder /preorder/postorder).
- ❑ Therefore, given a traversal it is not possible to recreate the underlying binary tree.
- ❑ But if for a binary tree both inorder and preorder traversals are given then it is possible to recreate the tree from those traversals.
- ❑ In other words two different binary trees may not have same inorder traversals and same preorder traversals.
- ❑ Same is the case if you are given inorder and postorder traversal.
- ❖ Question is - How to construct a binary tree from inorder and postorder (or preorder) traversal ?

Construct a binary tree from inorder and preorder (or postorder) traversal

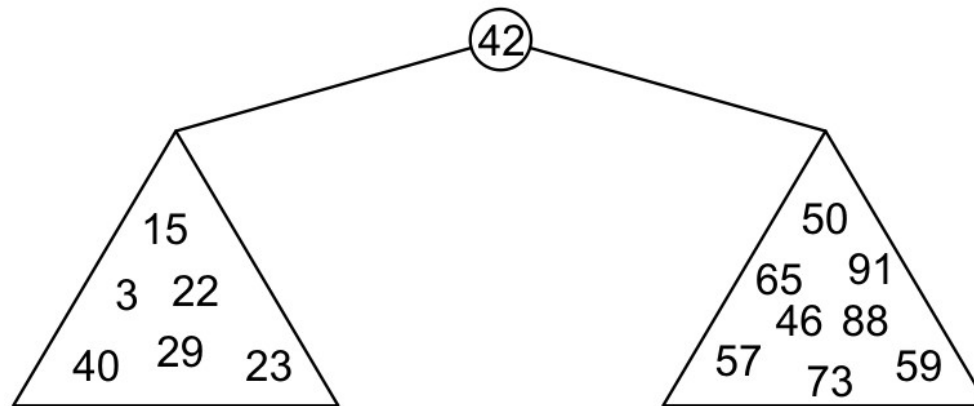
- ❑ Let us see the process of constructing a tree from
 - ❖ Inorder = {D G B A H E I C F} and
 - ❖ Preorder = {A B D G C E H I F}
- ❑ We first find the first node in Preorder (*or last node of Postorder*). The first node is "A", we know this value is the root as the root always appears at the first of preorder traversal.
- ❑ We search "A" in Inorder to find the left and right subtrees of the root. Everything on the left of "A" in Inorder is in the left subtree and everything on right is in the right subtree.
- ❑ Lets recur the above process for the following two:
 - Recur for Inorder = {D G B} and Preorder = {B D G}
 - ✓ Make the created tree as left child of root.
 - Recur for Inorder = {H E I C F} and Preorder = {C E H I F}
 - ✓ Make the created tree the right child of the root.



Binary Search Tree (BST)

Definition

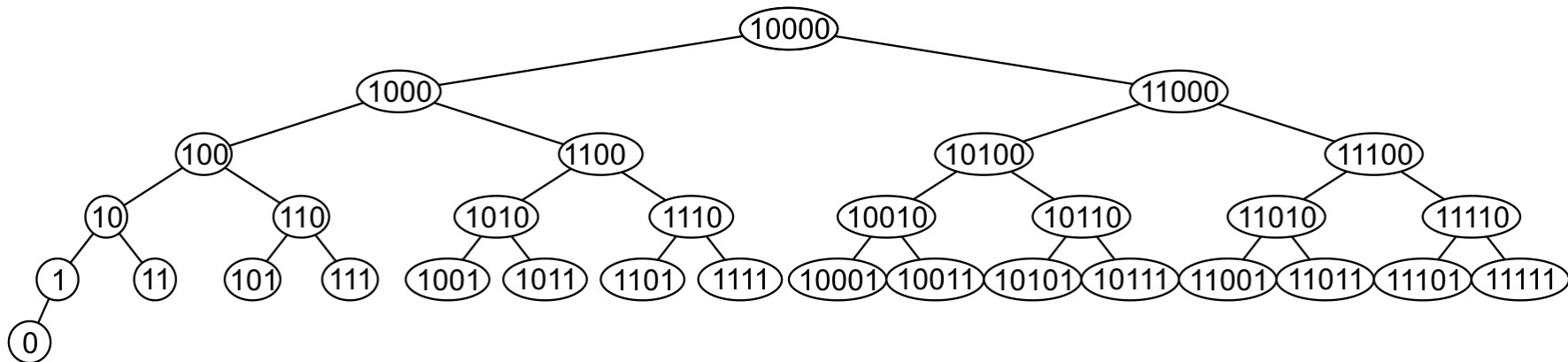
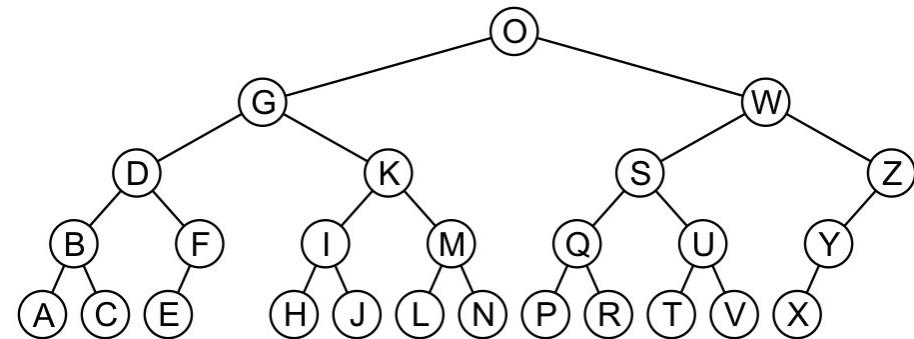
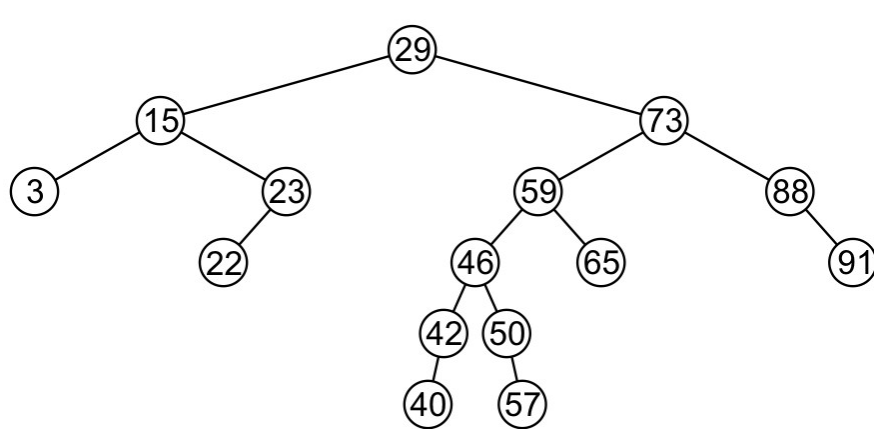
- ❑ A non-empty binary search tree as a binary tree with the following properties:
 1. The left sub-tree (if any) is a binary search tree and all values are less than the root value, and
 2. The right sub-tree (if any) is a binary search tree and all values are greater than the root value



❖ Duplicate values: assume that in any binary tree, we are not storing duplicate values unless otherwise stated. However, duplicate values can be considered with modifications to the algorithms we will cover.

Examples

❖ Examples of binary search trees:



Searching in a BST

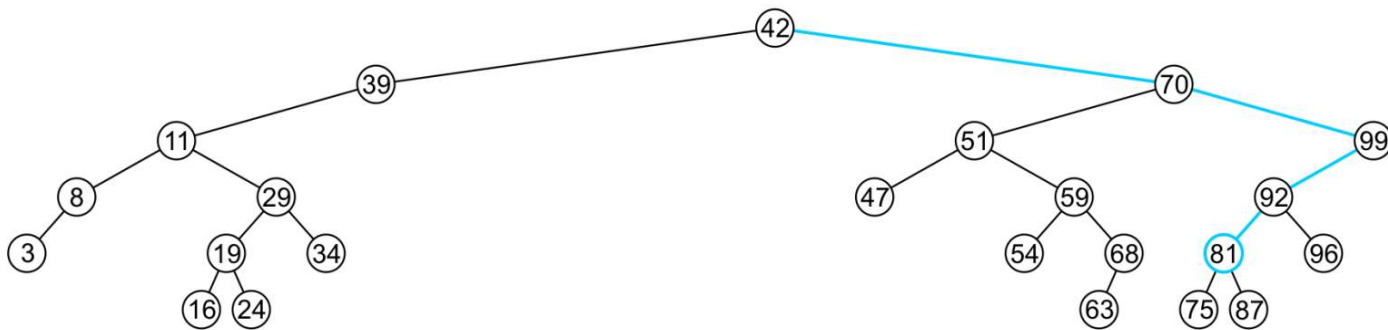
- ❑ Searching Technique: examine the root node and if we have not found what we are looking for:
 - If the object is less than what is stored in the root node, continue searching in the left sub-tree
 - Otherwise, continue searching the right sub-tree
- ✓ With a linear order, one of the following three must be true:

$$a < b \quad a = b \quad a > b$$

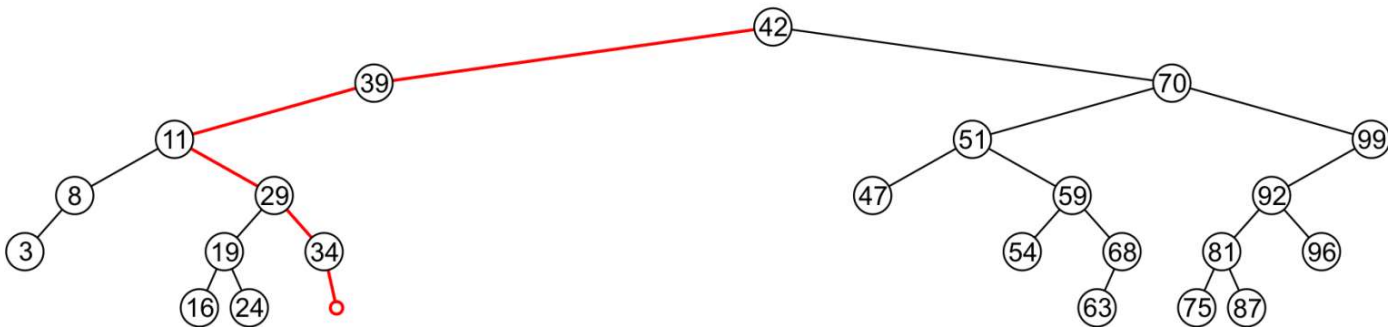
Searching

❑ To determine membership, traverse the tree based on the linear relationship:

❖ If a node containing the value is found, *e.g.*, 81, return 1



❖ If an empty node is reached, *e.g.*, 36, the object is not in the tree:



Searching: Subroutine

```
/* Iterative solution */
```

```
treenode * search(treenode *t, char key)
{
    treenode *tmp = t;
    bool found = false;
    while(tmp != NULL && !found) {
        if( key < tmp->element )
            tmp = tmp->lchild;
        else if(key > tmp->element )
            tmp = tmp->rchild;
        else // matching with key
            found = true;
    }
    return tmp;
}
```

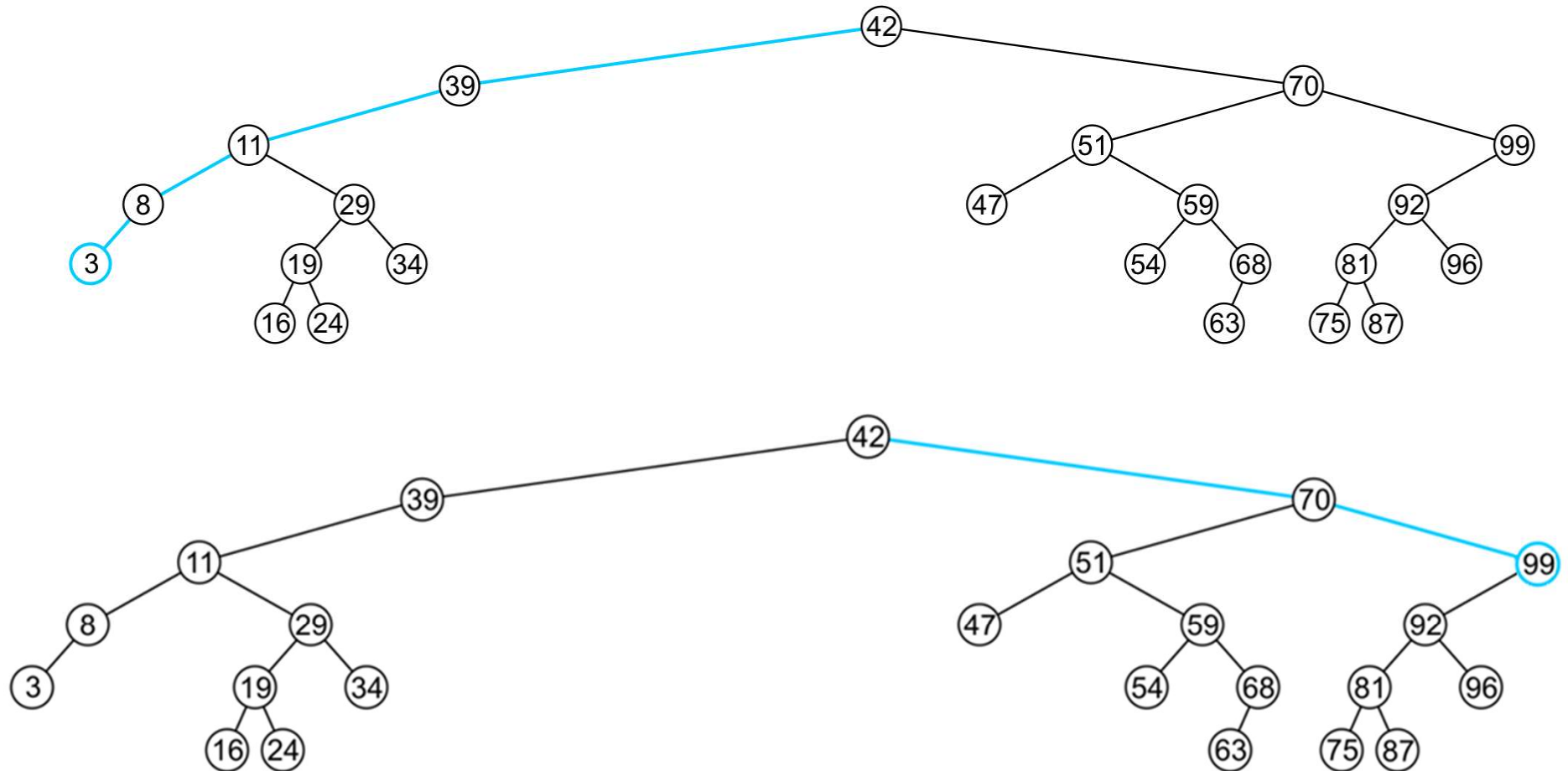
```
typedef struct node {
    char element;
    struct node *lchild;
    struct node *rchild;
    struct node *parent;
} treenode;
```

```
/* Recursive solution */
```

```
treenode * search(treenode *t, char key)
{
    treenode *tmp = t;
    if (tmp == NULL )
        return NULL;

    if (key < tmp->element)
        return search(tmp->lchild, key);
    else if(key > tmp->element )
        return search(tmp->rchild, key);
    else // matching with key
        return tmp;
}
```

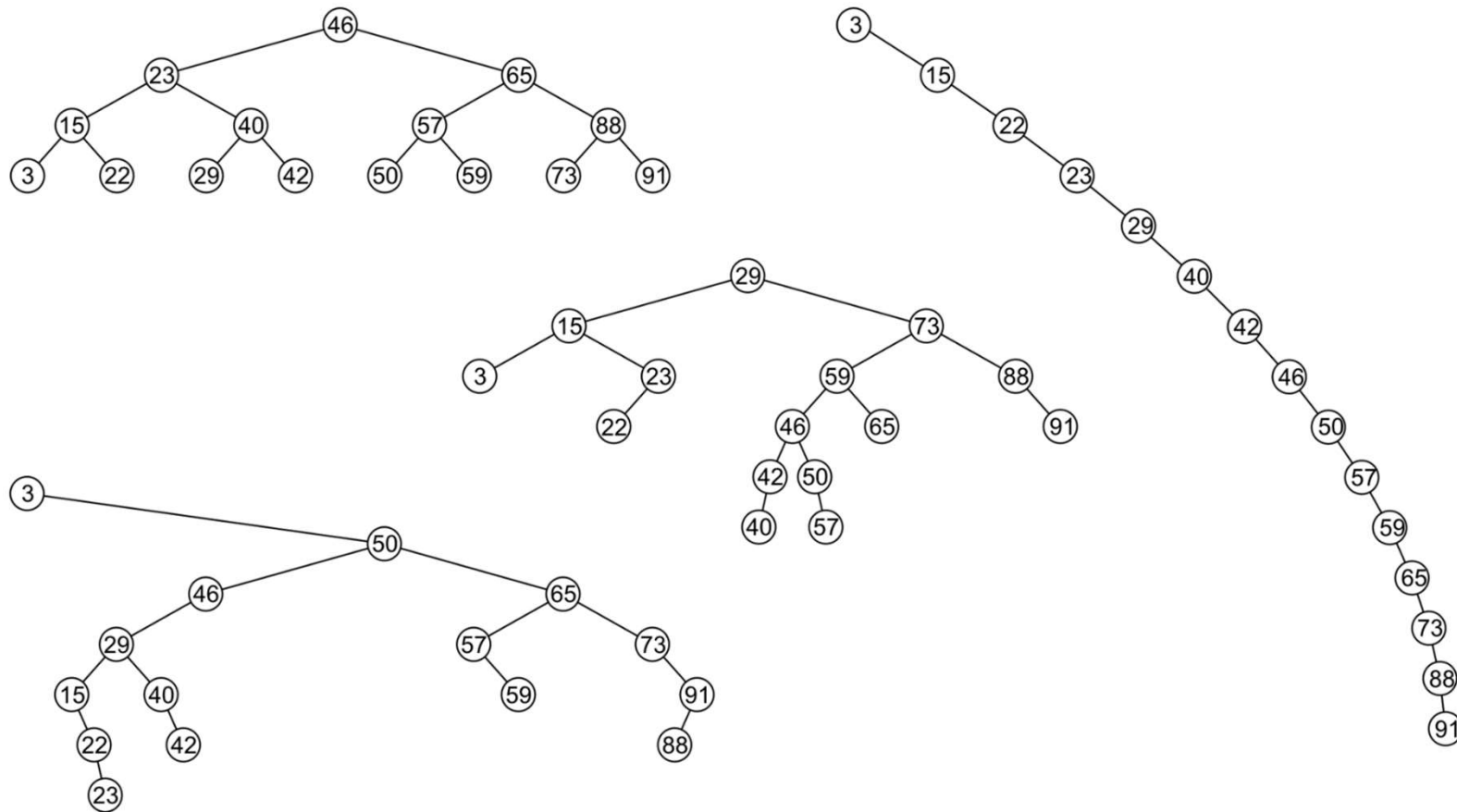
Finding the Minimum and Maximum element



Note: The extreme values are not necessarily leaf nodes

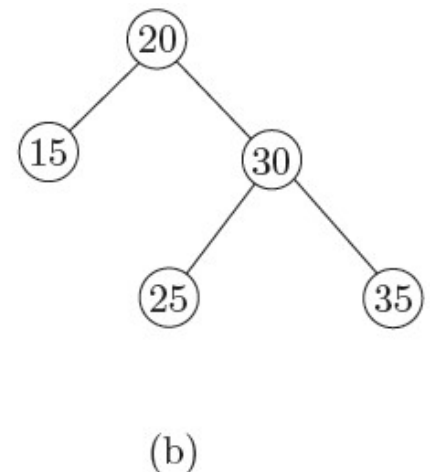
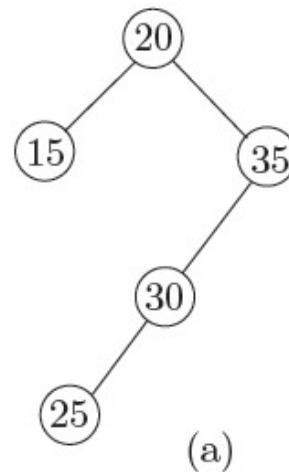
BST storing same data

❑ Following BSTs store the same data – **How?**



BST storing same data

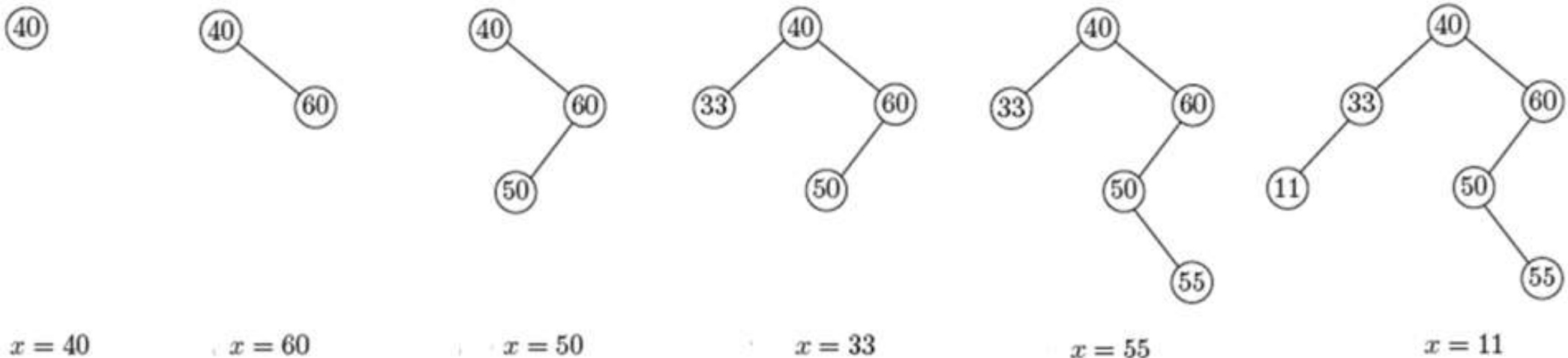
- ❑ Shape of binary search tree depend on the order of data.
- ❖ For example, two possible binary search trees for set of identifiers $\{15, 20, 25, 30, 35\}$ are shown in figures (a) and (b).
- Possible order of data for (a) could be $[20, 15, 35, 30, 25]$
- Possible order of data for (b) could be $[20, 15, 30, 25, 35]$



Inserting into a Binary Search tree

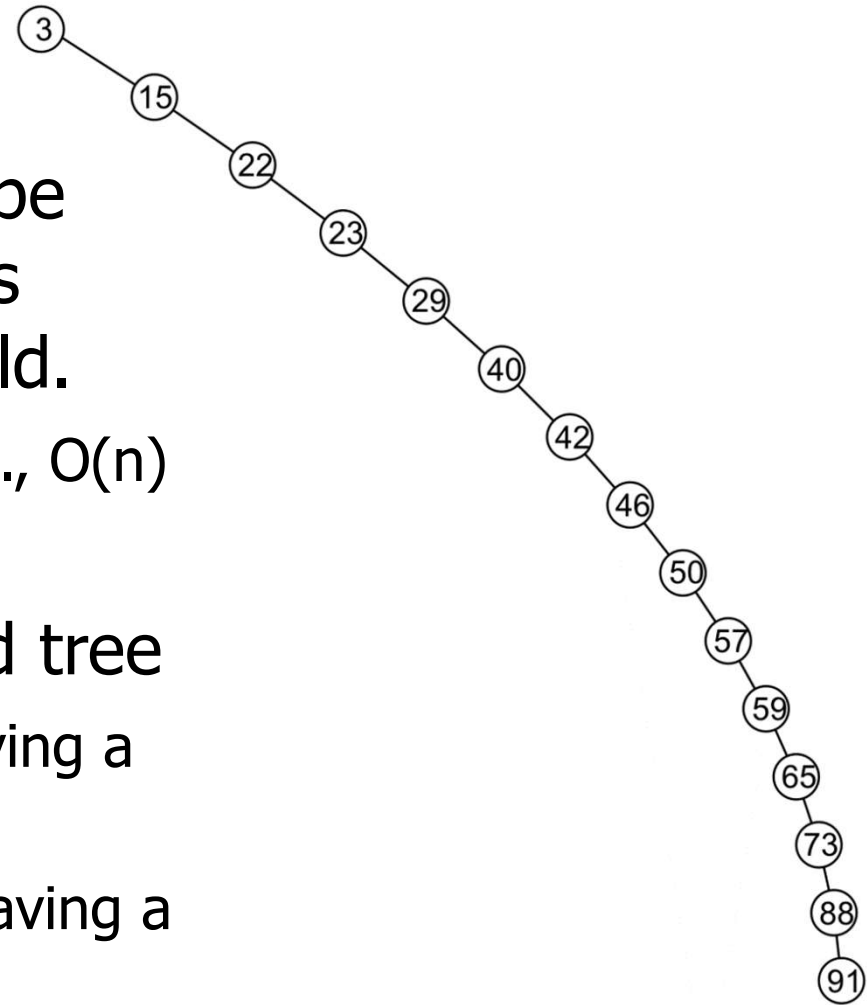
❑ An insertion will be performed at a leaf node:

❖ Example: Inserting a set of elements in the following order {40, 60, 50, 33, 55, 11} in an empty BST



Degenerate BST

- ❑ Unfortunately, it is possible to construct *degenerate* binary search trees
- ❑ A *skewed* binary search tree is a type of binary tree in which all the nodes have only either one child or no child.
 - This is equivalent to a linked list, i.e., $O(n)$
- ❑ There are 2 special types of skewed tree
 - Left Skewed - all the nodes are having a left child or no child at all
 - Right Skewed - all the nodes are having a right child or no child at all



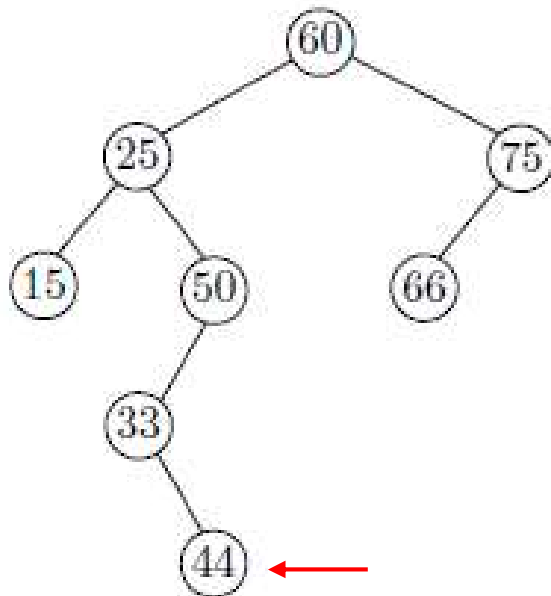
Deleting from a Binary Search tree

- ❑ A node being erased is not always going to be a leaf node
 - ❑ There are three possible scenarios:
 - The node is a leaf node,
 - It has exactly one child, or
 - It has two children (it is a full node)
-

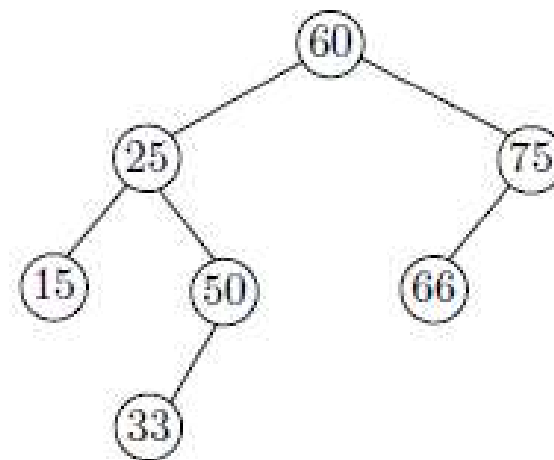
Deleting from a Binary Search tree - Case 1

❑ Case 1: The node to be deleted is the **leaf node**. In such a case, simply delete the node from the tree.

❖ Example: Deletion of 44 from the tree



(a) Before deletion



(b) After deletion

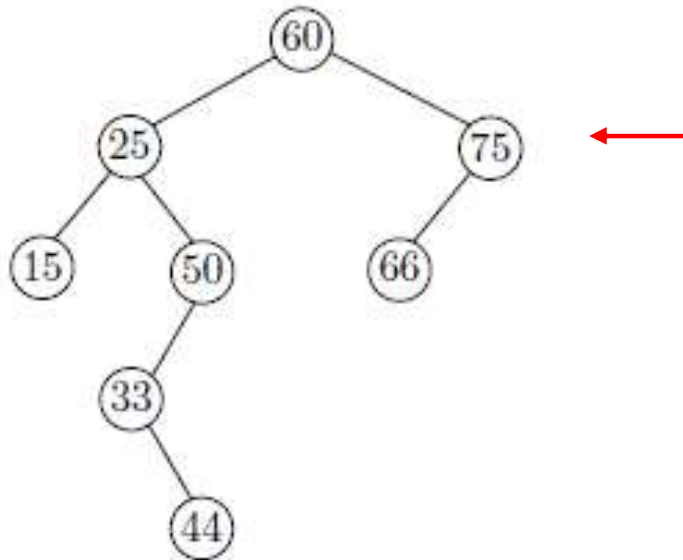
Deleting from a Binary Search tree - Case 2

❑ Case 2: The node to be deleted has a **single child node**.

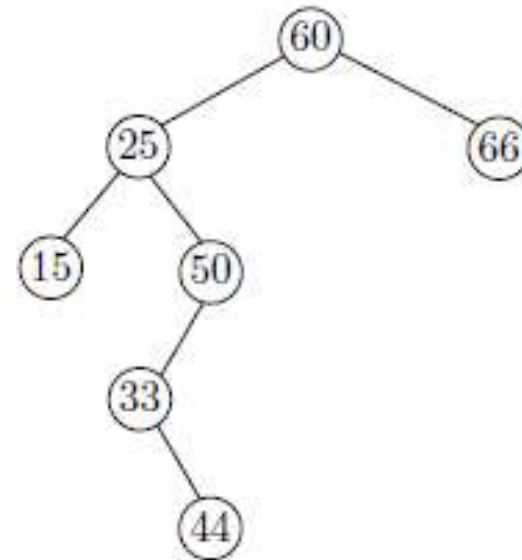
In such a case follow the steps below:

1. Replace that node with its child node.
2. Remove the child node from its original position.

❖ Example: Deletion of 75 from the tree



(a) Before deletion



(a) After deletion

Deleting from a Binary Search tree - Case 3

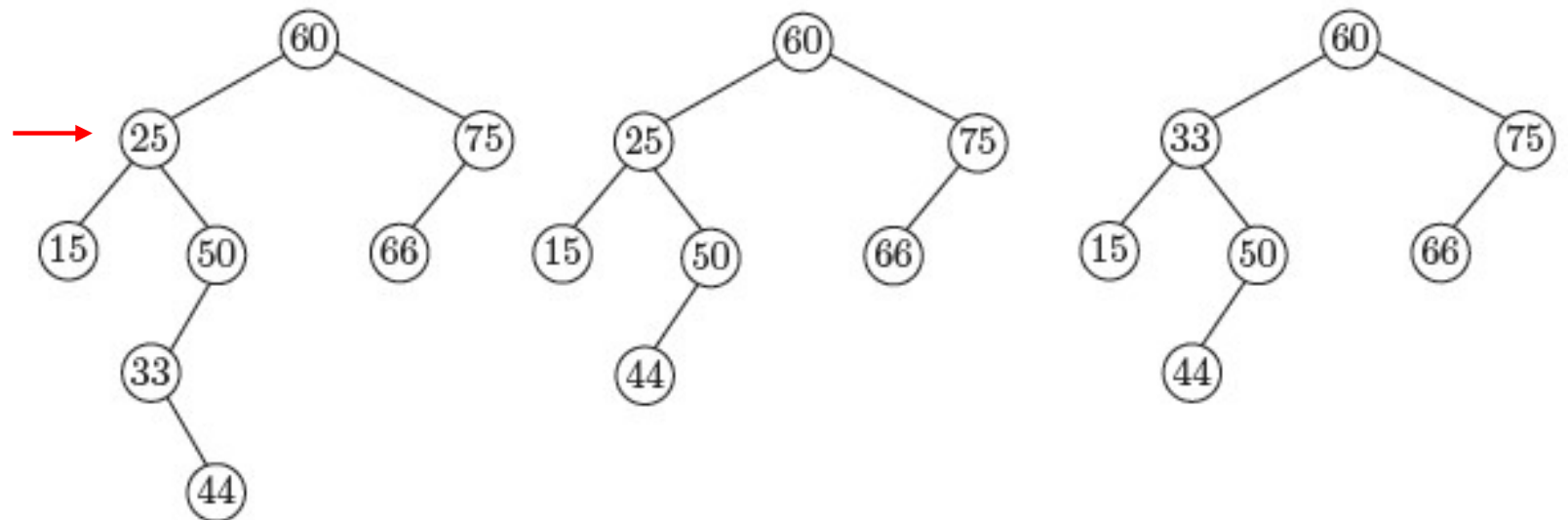
❑ Case 3: The node to be deleted has **two children**. In such a case follow the steps below:

1. Get the **inorder successor** (minimum object in the right sub-tree) of that node.

❖ **Note:** Inorder predecessor (maximum object in the left sub-tree) can also be used.

2. Remove the inorder successor from the tree, save the node information.
3. Replace the target node with the inorder successor node.

❖ Example:
Deletion of 25
from the tree



(a) Before deletion

(b) Temporary deletion of 33

(c) After deletion of 25

Run Time: $O(h)$

- ❑ Almost all of the relevant operations on a binary search tree are $O(h)$
 - If the tree is *close* to a linked list (skewed tree), the run times is $O(n)$
 - The best we can do is if the tree is perfect: $O(\log(n))$
 - Our goal will be to find tree structures where we can maintain a height of $O(\log(n))$
 - ❑ There are AVL trees, B+ trees
 - Both of which ensure that the height remains close to $O(\log(n))$
-

Acknowledgement

Most slides courtesy : Douglas Wilhelm Harder,
MMath, UWaterloo
